European Journal of Operational Research 292 (2021) 43–59

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

# A new algorithm for resource-constrained project scheduling with breadth and depth of skills

Jakob Snauwaert[a], Mario Vanhoucke[a,b,c,*]

[a] Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, Ghent 9000, Belgium
[b] Operations and Technology Management Centre, Vlerick Business School, Reep 1, Ghent 9000, Belgium
[c] UCL School of Management, University College London, 1 Canada Square, London E14 5AA, UK

ARTICLE INFO

*Article history:*
Received 8 April 2020
Accepted 26 October 2020
Available online 3 November 2020

*Keywords:*
Project scheduling
Resource-constrained scheduling
Multi-skilled resources
Genetic algorithm

ABSTRACT

This paper addresses a multi-skilled extension of the resource-constrained project scheduling problem (RCPSP). Although a handful of papers dealt with the multi-skilled RCPSP (MSRCPSP), little to no attention is given to the ideal levels of skills for multi-skilled resources. In this paper, skills are measured along two dimensions known as breadth and depth. In a project environment, the breadth of a resource is perceived as the amount of skills an employee masters. The depth of a skill is the efficiency level at which work can be performed by a resource that masters that skill. The MSRCPSP with breadth and depth consists of scheduling activities with skill requirements and assigning multi-skilled resources to those activities. To be able to efficiently solve the MSRCPSP, a genetic algorithm is developed. Using the created activity schedules and resources assignments, the best workforce characteristics are analysed. Key aspects in this analysis are the breadth and depth. The problem-specific procedure combines a new representation, a new crossover and tailor-made local searches. Computational experiments measure the impact of different multi-skilled resources and their efficiency levels on the makespan of the project.

© 2020 Elsevier B.V. All rights reserved.

## 1. Introduction

In recent years, corporations have been downsizing on their pool of resources (Datta, Guthrie, Basuil, & Pandey, 2010). This decrease in workforce size caused a new increasing demand in flexibility and versatility of workers. On top of that, organisations are always searching for ways to shorten their production runs and product cycle times, for which resources have to regularly acquire new skills and competences (Nembhard & Uzumeri, 2000). Workforces have become leaner and consist of cross-trained, or multi-skilled, workers, which master a diversified set of skills that can be used to execute a versatile range of activities (McCune, 1994). This new focus on flexible workers is induced by the long-term strategic advantages against competitors they generate (Gutjahr, Katzensteiner, Reiter, Stummer, & Denk, 2010). As multi-skilled resources are part of the workforce in a lot of projects (Haas, Rodriguez, Glover, & Goodrum, 2001), this study will analyse their effect on projects and their components.

This paper handles a multi-skilled extension of the resource-constrained project scheduling problem (RCPSP), which is NP-hard (Blazewicz, Lenstra, & Kan, 1983). The RCPSP has been one of the most researched problems in project scheduling literature. In particular, its many variants have been getting a lot of attention (Hartmann & Briskorn, 2010). In the RCPSP, activities are scheduled and resources are assigned to these activities with, in most studies, time or cost minimisation as an objective. In this case, the resources are assumed to master the complete skillset and are able to be assigned to every activity. In contrast, the workforce in the *multi-skilled resource-constrained project scheduling problem*, further abbreviated as MSRCPSP, has a heterogeneous skill-mix, and the multi-skilled resources do not master all the skills to execute every activity of the project. Instead, they are only available to activities that require their specific skills. In order to efficiently employ multi-skilled workers, the goal of this study is to find the ideal size of the workforce and the corresponding skill levels of the resources.

In this extension to the problem, the multi-skilled aspect is characterised by two concepts known as breadth and depth (Gomar, Haas, & Morton, 2002). Breadth represents understanding the different components of the project and recognising how they fit together. A person with a high breadth is someone who is able to keep an overview of the tasks at hand. In a project envi-

* Corresponding author at: Faculty of Economics and Business Administration, Ghent University, Tweekerkenstraat 2, Ghent 9000, Belgium.
*E-mail addresses:* jakob.snauwaert@ugent.be (J. Snauwaert), mario.vanhoucke@ugent.be (M. Vanhoucke).

https://doi.org/10.1016/j.ejor.2020.10.032
0377-2217/© 2020 Elsevier B.V. All rights reserved.

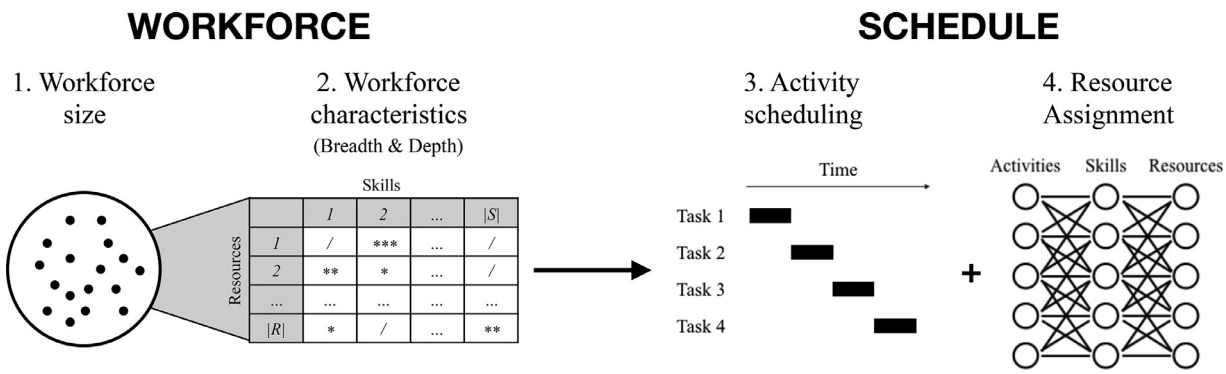


**Fig. 1.** Multi-skilled resources in project scheduling.

ronment, the breadth is perceived as the amount of skills the resource masters. Depth refers to a well-developed understanding as well as a higher expertise an employee can have for a skill. Consequently, depth is translated into the efficiency level of a resource for a certain skill, i.e. the level of depth for a skill defines how long it takes the resource to finish the activity in a project. In this study, we aim at extending the current models for the MSRCPSP with the previously discussed breadth and depth dimensions. Although, papers such as Bellenguez and Néron (2004), Valls, Pérez, and Quintanilla (2009), Heimerl and Kolisch (2010), Yannibelli and Amandi (2011) and Kolisch and Heimerl (2012) implicitly incorporate the breadth and depth of resources in order to manage multi-skilled resources in projects, none of them address (1) the determination of ideal levels of breadth for the resources or (2) the impact of depth of multiple assigned resources on the activity durations. When considering multi-skilled resources in a project scheduling environment, there are two important problems (see Fig. 1). The first problem that will be analysed is the determination of the ideal size and characteristics of the workforce. For various projects, we will investigate the impact of the number of workers and their breadth and depth on the quality of the schedules. Moreover, we will analyse the differences between a workforce with a large skillset (focus on breadth) and a diverse workforce with experienced and novice employees (focus on depth). Secondly, we will solve the MSRCPSP and thus construct resource-feasible project schedules. More specifically, we will search for satisfactory activity schedules and investigate the impact of skills on the project. The objective is to assign multi-skilled resources to the activities to generate feasible solutions for the MSRCPSP that minimise the makespan.

The outline of this paper is as follows. Section 2 gives an overview of the application of skills in the project scheduling literature. In Section 3 our view on the MSRCPSP is described, while Section 4 proposes a genetic algorithm to efficiently solve this problem. The computation results are analysed and explained thoroughly in Section 5. In the last section, the paper ends with conclusions and further research opportunities.

## 2. Literature review on multi-skilled resources

Incorporating multiple skills creates a number of important advantages. According to Mansfield (1996) there are less career opportunities for workers in today's lean organisational structure. Therefore, multi-skilling is essential for the workers growth inside the organisation. With a broadened skillset workers can be employed in unanticipated sectors and create sustainable strategic positions (Gutjahr et al., 2010). Corporations can now reduce turnover, which will help the job security of the workers (Gomar et al., 2002). A multi-skilled workforce will, ceteris paribus, be more productive and balanced, in terms of workload, than a workforce with single-skilled workers (Hopp, Tekin, & Van Oyen, 2004). Furthermore, multi-skilled workers make it easier to overcome absenteeism (Inman, Jordan, & Blumenfeld, 2004) and reduce the amount of idle time (Martignago, Battaïa, & Battini, 2017).

Despite these positive benefits, the use of multi-skilled resources might also evoke negative consequences. Most notably, the wage cost of a multi-skilled worker is elevated in comparison to a single-skilled worker (Cai & Li, 2000). The procurement costs are higher because it is more intricate to find workers with the right set of skills on the labor market (Gutjahr et al., 2010). Training multi-skilled employees is also more costly, even if it is handled internally in the organisation (Inman et al., 2004). Additionally, multi-skilling can cause negative behavioural effects which lower the productivity (Schultz, McClain, & Thomas, 2003) and can reduce the quality of performed work due to overskilling (Pinker & Shumsky, 2000).

In Table 1, we display a list of studies that dealt with multi-skilled resources in project scheduling problems. The table shows the objective and the method used in each study. Moreover, the used dataset and some specifications of the dataset are presented in the next columns. Only some other studies included varying efficiency levels in their studies. The last column explains if and how the study dealt with different efficiency levels. In this paper, efficiency of skills is incorporated as the depth of the skills of resources.

Brusco and Johns (1998) were the first to consider multi-skilled resources with varying levels of efficiency. However, they solve the multi-skilled workforce scheduling problem, which only deals with workforce staffing and not with project scheduling. The multi-skilled project scheduling problem was first studied by Hegazy, Shabeeb, Elbeltagi, and Cheema (2000), who utilise resource substitution rules to create multi-skill opportunities. The substitution rules allow for $n$ resources of type $i$ to substitute $m$ resources of type $j$ to execute an activity. This implies that the whole workforce masters the complete skillset because each resource type can always be replaced by the other resource types, which is not the case in the current paper. Afterwards Néron (2002) replaced the resource substitution rules with resource skillsets. Moreover, a suggestion towards two lower bounds was shown. Bellenguez and Néron (2004) add the depth concept to the study of Néron (2002), which they call a hierarchical skill level. They incorporate the depth of skills as a way to distinguish different skill requirements, which indicates that the depth only has an impact on the staff assignment but not on the activity durations. Efficiency levels are also presented in Valls et al. (2009), where they have an impact on the task duration, however the tasks only require a single worker and their objectives are resource-oriented, i.e. urgency levels, assignment effectivity and balanced workload. Heimerl and Kolisch (2010) and Kolisch and Heimerl (2012) incorporate varying efficiencies that affect activity duration, however, this is done in a multi-project environment with some restrictions

**Table 1**
Overview of the literature on multi-skilled resources in project scheduling problems.

| Author(s) | Objective[a] | Method[b] | Dataset | $\|N\|$[c] | $\|R\|$[c] | $\|S\|$[c] | Efficiency levels |
|---|---|---|---|---|---|---|---|
| Hegazy et al. (2000)[d] | M | Heuristic | Case Study | 20 | 77 | / | / |
| Néron (2002) | M | Lower bound | / | / | / | / | / |
| Bellenguez and Néron (2004) | M | Lower bound | / | / | / | / | Skill Requirement |
| Bellenguez-Morineau and Néron (2007) | M | B&B | Own | 12–32 | 4–10 | 3–5 | / |
| Alba and Chicano (2007) | M/C | GA | Own | 30 | 5/10/15/20 | 2–10 | / |
| Li and Womer (2009) | C | HBD | Own(ProGen) | 10/30 | Varying | 4/8 | / |
| Valls et al. (2009)[e] | UL/BA/BW | Hybrid-GA | Own | 50/100/200[e] | 6–47 | 3 | Variable Duration |
| Heimerl and Kolisch (2010)[f] | C | MILP | Own | / | Varying | 4 | Variable Duration |
| Yannibelli and Amandi (2011) | E | GA | Own(ProGen) | 20/30/40/50 | Varying | 4 | Objective |
| Kolisch and Heimerl (2012)[f] | C | GA-TS | Own | 6 | 5/10 | 4 | Variable Duration |
| Correia et al. (2012) | M | MILP/Heuristic | Own(PSPLIB) | 20 | 10–30 | 4 | / |
| Liu and Wang (2012) | M | CP | Own | 5 repetitive | / | / | / |
| Correia and Saldanha-da Gama (2014) | C | MILP | Correia et al. (2012) | 20 | 10–30 | 4 | / |
| Montoya, Bellenguez-Morineau, Pinson, and Rivreau (2014) | M | B&P | Own(PSPLIB,...) | 20–62 | 2–19 | 2–15 | / |
| Almeida et al. (2016) | M | Heuristic | Correia et al. (2012)/Own | 20/40 | 10–60 | 4 | / |
| Almeida et al. (2018a) | M | GA | Correia et al. (2012)/Own | 20/40 | 10–60 | 4 | / |
| *Snauwaert and Vanhoucke* | M | Hybrid-GA | Own(RanGen) | 30 | 8–216 | 4 | Variable Duration |

[a] M = makespan; C = cost; UL = urgency levels; BA = best assignment; BW = balance workload; E = employee effectiveness.
[b] B&P = branch-and-price; B&B = branch-and-bound; HBD = hybrid MILP/CP Benders decomposition; MILP = mixed-integer linear programming; TS = tabu-search; CP = constraint programming; GA = Genetic Algorithm.
[c] $|N|$ = number of activities; $|R|$ = number of resources; $|S|$ = number of skills.
[d] Substitution rules instead of skills.
[e] Every activity only requires 1 resource.
[f] Part of multi-project problems.

on the project schedules. Yannibelli and Amandi (2011) use efficiency levels to maximise employee effectiveness. Recently, papers have been published that focus on solving the MSRCPSP. Some of them encapsulate the MSRCPSP in a linear programming model (Li and Womer (2009); Correia, Lampreia-Lourenço, and Saldanha-da Gama (2012); Almeida, Correia, and Saldanha-da Gama (2018b)). Moreover, various heuristic and exact solutions have been proposed for this problem. A hybrid Benders decomposition approach is developed in Li and Womer (2009) and Bellenguez-Morineau and Néron (2007) present a branch-and bound procedure. Almeida, Correia, and Saldanha-da Gama (2016) and Almeida, Correia, and Saldanha-da Gama (2018a) address the multi-skill project scheduling problem using respectively a heuristic framework and a biased random-key genetic algorithm. In the literature, multi-skilled resources in projects have been utilised in software development (Alba & Chicano, 2007), the service sector (Valls et al., 2009), the building sector (Liu & Wang, 2012) and consulting (Correia et al., 2012).

## 3. Problem description

In the MSRCPSP, first introduced by Bellenguez and Néron (2004), a project is represented by a topologically ordered acyclic activity-on-the-node network $G = (N, A)$, in which the set $A$ is used to represent the minimum finish-start precedence relations between activities with a time-lag of zero and the set $N$ represents the activities of the project, embodied by the nodes in the graph. The activities are numbered from a dummy start node 0 to a dummy end node $|N| + 1$, such that there are $|N|$ non-dummy nodes. The dummy nodes have a duration of zero and no resource usage. The activities are executed by a set of renewable multi-skilled resources $R$, with $R = \{1, ..., |R|\}$. Each resource $k \in R$ masters a number of skills from the set of skills $S$, with $S = \{1, \ldots, |S|\}$. Since each resource $k \in R$ masters different skills of the set $S$, we assume that the set of resources $R$ is considered to be human and therefore resources and workers are used interchangeably throughout this paper. Each activity $i \in \{1, \ldots, |N|\}$ requires $r_{ij}$ workers with skill $j \in S$. Hence, the resource demand of each activity is now expressed as a demand for skills and will be further referred to as the *skill requirements*. Fig. 2 displays how the resource requirements of the RCPSP are now modelled as skill requirements for the MSRCPSP. In the RCPSP each activity $i$ requires a certain number of workers of type $k \in \{1, \ldots, |K|\}$. The $r_{ik}$-matrix shows that, in this figure, 1 worker of type 1 and 0 workers of type 2 need to be assigned to activity 1. For the MSRCPSP assigning resources to activities is done in a different way, as follows, an activity $i$ requires a number of units of skill $j$ and each worker $k$ masters a set of skills $j \in S$ with which he/she can work on activity $i$. The $r_{ij}$-matrix shows that the activities require a number of skills instead of resources, activity 1 now requires 1 unit of skill 1 and 0 units of skill 2. The $d_{jk}$-matrix represents the efficiency levels (depth) of the workers and the skill distribution in the workforce. Both resource 1 and 2 can fulfil the requirement of activity 1 as they master skill 1 at an efficiency level of, respectively, 1 and 1.2 (i.e. resource 2 has a higher depth and is therefore more efficient). Hence, a resource is a worker $k$ who can only be assigned to an activity $i$ if it masters one of the skills required by the activity. Moreover, when this activity requires multiple different skills, a worker can satisfy exactly one of these skills. Since the amount of workers is limited, each skill $j \in S$ is also limited in availability. Consequently, the skill availability $a_j$ is equal to the number of workers that master skill $j$. The goal is to generate a baseline schedule, with starting times $s_i$ and finishing times $f_i$, and to assign multi-skilled resources to each activity $i \in N$, while minimising the makespan.

The relations between workers and skills are defined by two concepts. *Depth* $d_{jk}$ describes the efficiency level of resource $k$ for skill $j$. It is the efficiency with which the resource can perform a skill, which is related to the speed of a resource in executing the activity. The relation between the depth of a resource and the duration of an activity is presented below.

$$d_{jk} \begin{cases} \in \{d^{min}, \ldots, d^{max}\}, & \text{if resource } k \text{ masters skill } j, \text{ with } d^{min} > 0. \\ = 0, & \text{if resource } k \text{ does not master skill } j. \end{cases} \tag{1}$$

A resource with a standard efficiency level has a default value of 1. A more efficient resource $k$ is represented by a depth higher than 1, $d_{jk} > 1$. Hence, this resource can execute the same amount of work for skill $j$ in a shorter amount of time than a resource with the

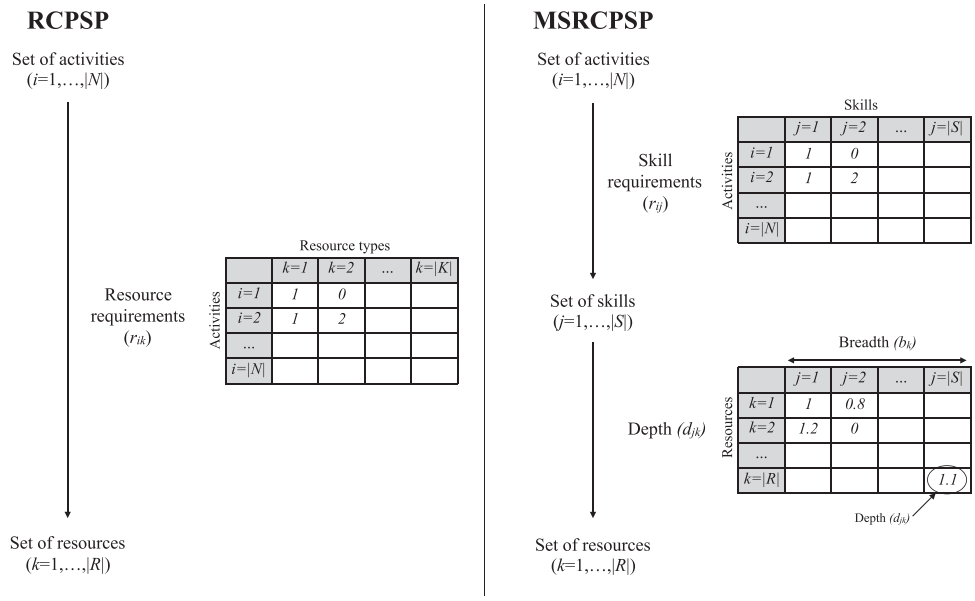


**Fig. 2.** Comparison between RCPSP and MSRCPSP.

default depth value. The opposite is true for less efficient resources, i.e. $d_{jk} < 1$. For every resource $k$ that does not master skill $j$ the depth $d_{jk}$ is equal to 0.

*Breadth* $b_k \in \{1, \ldots, |S|\}$ represents how many skills a resource $k$ masters. This $b_k$ can be derived from the depth $d_{jk}$. Each resource $k$ possesses an integer amount of skills, described by this breadth $b_k$, which is directly correlated to the flexibility of the resource. Based on the breadth, the set of multi-skilled resources $R$ can be divided into three sets, i.e. the set of single-skilled resources $R_s$, with $b_k = 1$, the set of completely-skilled resources $R_c$, with $b_k = |S|$ and the set of multi-skilled resources $R_m$, to refer to resources that master a number of skills in the $[2, |S| - 1]$ interval. Obviously, $R_s \cap R_c \cap R_m = \emptyset$ and $R_s \cup R_c \cup R_m = R$.

### 3.1. Activity durations

Every activity $i \in N$ has a predefined non-preemptive processing time $p_i$, which coincides with the time it would take resources $k \in R$ with skills of the standard depth level $d_{jk} = 1$ to finish the activity. Considering the depth $d_{jk}$ of every resource $k$ with skill $j$ assigned to activity $i$, a new adjusted processing time $p_i^a$ is calculated. Based on an extensive analysis (see Technical Appendix), we propose a reciprocal relation between the depth of the assigned resources and the adjusted duration of the activity:

$$p_i^a = p_i \cdot \max_j \left( \frac{r_{ij}}{\sum_{k \in R} d_{jk} \cdot x_{ijk}} \right) \tag{2}$$

with $x_{ijk}$ equal to 1 if resource $k$ works on activity $i$ with skill $j$ and 0 otherwise. The nominator of the fraction in Eq. (2) is equal to the number of resources required by skill $j$ in activity $i$ and the denominator calculates the total summed depth per skill $j$ in activity $i$. Hence, the latter part of this equation calculates the reciprocal of the average assigned depth to each skill $j$. Since the adjusted processing time $p_i^a$ will depend on the skill $j$ that is performed the slowest (i.e. with the smallest average assigned depth), we multiply the standard processing time $p_i$ with the maximal reciprocal of the average assigned depth to activity $i$. Moreover, this equation implies that a high depth resource of skill $j$ can compensate for a low depth resource of skill $j$, but resources performing a different skill $j'$ have no impact on the resources performing skill $j$. For example, if the average assigned depths for an activity with 2 skills are equal to 0.5 for skill 1 and 1.5 for skill 2, then the $p_i^a$ will only depend on the average depth of skill 1, because it is the lowest. The reciprocal of this smallest averaged assigned depth is equal to 2 (=1/0.5) and therefore the adjusted duration $p_i^a$ of the activity will be twice as long as the standard duration $p_i$.

Table 2 shows a number of papers that dealt with breadth and depth of skills and variable activity durations. Valls et al. (2009) were the first to consider depth and variable activity durations in a multi-skilled project scheduling environment. In their case, more (less) efficient workers have a depth lower (higher) than 1 and each activity requires a single worker. The depth of the worker is then multiplied by the standard processing time to determine the adjusted processing time in a linear manner. Heimerl and Kolisch (2010) and Kolisch and Heimerl (2012) presented a reciprocal relation between efficiency and working speed in a multi-project multi-skilled project scheduling problem. However, they disregarded activity durations and used the depth of the resources only to restrict the regular and overtime working capacity of resources. A reciprocal relation between depth and activity duration has also been proposed in papers that deal with workforce scheduling. In Walter and Zimmermann (2016) multiple resources can be assigned to an activity, as is the case in this problem. To represent the relationship between the depth of multiple resources and the eventual task duration, they also use the reciprocal of the average depth of the assigned resources.

### 3.2. Mathematical model (Table 3)

$$\text{Min} \quad s_{|N|+1} \tag{3}$$

$$\text{s.t.} \quad \sum_{k \in R} x_{ijk} = r_{ij} \qquad \forall i \in N \setminus \{0, |N|+1\}, j \in S \tag{4}$$

$$s_i + p_i^a - M(1 - z_{ii'})(1 - Prec_{ii'}) \le s_{i'} \qquad \forall i, i' \in N \tag{5}$$

$$z_{ii'} + z_{i'i} \le 1 \qquad \forall i, i' \in N : i < i' \tag{6}$$

$$\sum_{j \in S} x_{ijk} + \sum_{j \in S} x_{i'jk} \le 1 + z_{ii'} + z_{i'i} \qquad \forall i, i' \in N : i < i', k \in R \tag{7}$$

$$\sum_{j \in S} x_{ijk} \le 1 \qquad \forall i \in N \setminus \{0, |N|+1\}, k \in R \tag{8}$$

**Table 2**
Papers in the literature that deal with breadth and depth of skills and variable durations.

| Paper | Subject[a] | # resources | # of discrete levels | Depth range | Impact |
|---|---|---|---|---|---|
| Valls et al. (2009) | PS | 1 | 3 | {0.75,1,1.25} | Linear |
| Heimerl and Kolisch (2010) | MPS | ≥ 1 | / | [0.5,1.5] | Reciprocal |
| Kolisch and Heimerl (2012) | MPS | ≥ 1 | / | [0.5,1.5] | Reciprocal |
| Walter and Zimmermann (2016) | WS | ≥ 1 | 4 | {0.5,1,1.5,2} | Reciprocal |
| Snauwaert and Vanhoucke | PS | ≥ 1 | 5 | {0.5,0.75,1,1.25,1.5} | Reciprocal |

[a] PS = project scheduling, MPS = multi-project scheduling, WS= workforce scheduling.

**Table 3**
Mathematical model details.

| | | |
|---|---|---|
| Parameters | $r_{ij}$ | Number of resources required with skill $j$ by activity $i$ |
| | $Prec_{ii'}$ | 1, if activity $i$ precedes activity $i'$, 0, otherwise |
| | $p_i$ | Standard processing time of activity $i$ |
| | $d_{jk}$ | Depth of resource $k$ for skill $j$ |
| | $d^{min}$ | Minimum possible depth |
| | $y_v$ | Data point for $d_i$ on breakpoint $v$ of the piecewise linear function |
| | $f(y_v)$ | Data point for $f(d_i)$ on breakpoint $v$ of the piecewise linear function |
| Sets | $N$ | Set of activities |
| | $R$ | Set of resources |
| | $S$ | Set of skills |
| | $V$ | Set of breakpoints of the piecewise linear function |
| Decision Variables | $x_{ijk}$ | 1, if resource $k$ is assigned to skill $j$ of activity $i$, 0, otherwise |
| | $z_{ii'}$ | 1, if activity $i$ is finished before activity $i'$ starts, 0, otherwise |
| | $s_i$ | Start time of activity $i$ |
| | $p_i^a$ | Adjusted processing time of activity $i$ |
| | $d_i$ | Depth of activity $i$ |
| | $f(d_i)$ | Reciprocal of depth of activity $i$ |
| | $\alpha_{iv}$ | Value for each breakpoint $v$ such that $d_i = \sum_{v \in V} \alpha_{iv} \cdot y_v$ |
| | $\beta_{iv}$ | 1, if $f(d_i)$ is calculated from the $v$th segment for activity $i$, 0, otherwise |

$$p_i^a = p_i \cdot \max_j \left( \frac{r_{ij}}{\sum_{k \in R} d_{jk} \cdot x_{ijk}} \right) \qquad \forall i \in N \tag{9}$$

$$x_{ijk} \leq d_{jk} + (1 - d^{min}) \qquad \forall i \in N, j \in S, k \in R \tag{10}$$

$$s_i \geq 0 \qquad \forall i \in N \tag{11}$$

$$z_{ii'} \in \{0, 1\} \qquad \forall i, i' \in N \tag{12}$$

$$x_{ijk} \in \{0, 1\} \qquad \forall i \in N, j \in S, k \in R \tag{13}$$

In the objective function (3), the makespan is minimised. Constraints (4) address the skill requirements for each individual activity. Constraints (5) make sure that the precedence constraints are fulfilled. Parameter $M$ represents a large value. Constraint set (6) display the logical constraints for the $z_{ii'}$ variables. Constraints (7) guarantee that a single resource can not be assigned to parallel scheduled activities. To ensure that every resource can only fulfil one of the skill requirements per activity, constraints (8) are implemented. Eq. (9) calculate the adjusted processing time. Next, constraints (10) establish that employees can only perform a skill if they master it, which holds true if $d_{jk} > 0$. Lastly, constraints (11), (12) & (13) define the decision variables.

The non-linear character of Eq. (9) is compatible with the genetic algorithm that is presented in Section 4. However, in Section 5, we compare this new procedure with a MILP model solved in CPLEX. To that purpose we linearise Eq. (9) using piecewise linear approximation. If we introduce a new variable $d_i$ $(= \min_j(\frac{\sum_{k \in R} d_{jk} \cdot x_{ijk}}{r_{ij}}))$, we can further simplify this equation to (14) to facilitate this linear approximation.

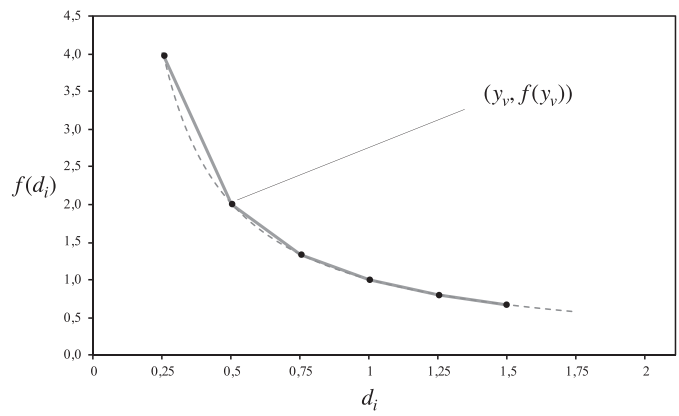


**Fig. 3.** Piecewise linear approximation.

$$p_i^a = p_i \cdot \frac{1}{d_i} \tag{14}$$

The function to be linearised $f(d_i)$ contains one decision variable $d_i$. $f(d_i)$ is the reciprocal of $d_i$ and is approximated by introducing sampling coordinates $y_v$ with $v \in V$. Correspondingly, the linear segments $[(y_v, f(y_v)), (y_{v+1}, f(y_{v+1}))]$ are used to estimate $f(d_i)$ (Fig. 3). For any given value of $d_i$, $f(d_i)$ is approximated by a convex combination of $f(y_v)$ and $f(y_{v+1})$. To put this into the mathematical model, we introduce a binary dummy variable $\beta_{iv}$, that represents the $v^{th}$ segment of the piecewise linear function. $\beta_{i0}$ and $\beta_{i|V|}$ are the extreme points, which are, respectively, equal to $d_{min}$ and $d_{max}$. Similarly, $\alpha_{iv} \in [0, 1]$ is a continuous dummy variable for each breakpoint $v$. The approximation of $f(d_i)$ can be accomplished by substituting Eq. (9) in the mathematical model by the following constraints:

$$d_i \leq \frac{\sum_{k \in R} d_{jk} \cdot x_{ijk}}{r_{ij}} \qquad \forall i \in N, j \in S \tag{15}$$

$$\sum_{v \in V} \beta_{iv} = 1 \qquad \forall i \in N \tag{16}$$

$$\sum_{v \in V} \alpha_{iv} = 1 \qquad \forall i \in N \tag{17}$$

$$\alpha_{iv} \leq \beta_{iv-1} + \beta_{iv} \qquad \forall i \in N, v \in V \setminus \{0\} \tag{18}$$

$$\alpha_{i0} \leq \beta_{i0} \qquad \forall i \in N \tag{19}$$

$$d_i = \sum_{v \in V} \alpha_{iv} \cdot y_v \qquad \forall i \in N \tag{20}$$

$$f(d_i) = \sum_{v \in V} \alpha_{iv} \cdot f(y_v) \qquad \forall i \in N \tag{21}$$

$$p_i^a = p_i \cdot f(d_i) \qquad \forall i \in N \tag{22}$$

Constraints (15) specify that we use the average depth of the least efficient skill to calculate the reciprocal relation. Eq. (16) assure that only 1 linear segment is chosen from which $f(d_i)$ will be estimated. Next, Eq. (17) impose that the sum of the continuous variables are equal to 1 such that $d_i$ and $f(d_i)$ are a combination of breakpoints. Constraints (18) and (19) guarantee that if $\beta_{iv} = 1$, only $\alpha_{iv}$ and $\alpha_{iv+1}$ can be different from 1. Eqs. (20) and (21) calculate $d_i$ and $f(d_i)$ as a combination of breakpoints on the piecewise linear function. Finally, Eq. (22) calculate the new adjusted processing time $p_i^a$ for each activity $i$ based on the standard duration $p_i$ and the reciprocal of the average assigned depth $f(d_i)$.

## 4. Solution approach

The multi-skilled resource-constrained project scheduling problem under study is solved by a meta-heuristic search algorithm. The outline of this section is as follows. First, three key criticality metrics are proposed which are applied in various parts of the solution procedure. Next, it presents a genetic algorithm (GA) and the different steps of the algorithm. The different parts of the solution approach are further explained in a detailed example of the complete solution approach that will be made available online (www.projectmanagement.ugent.be/research/project_scheduling/MSRCPSP).

### 4.1. Criticalities

**Skill criticality:** The *skill criticality* is a measure that quantifies how crucial each skill is to the project and is derived from the one presented in Bellenguez-Morineau and Néron (2007). More specifically, the higher the criticality of a skill, the higher the chance that this skill will cause resource conflicts. The skill criticality $SC_j$ of skill $j$ is:

$$SC_j = \frac{\sum_i (r_{ij} \cdot p_i)}{\sum_k y_{jk} \cdot \sum_i p_i} \tag{23}$$

with binary variable $y_{jk}$ equal to 1 if resource $k$ masters skill $j$ and zero otherwise,

$$y_{jk} = \begin{cases} 1, & \text{if } d_{jk} > 0. \\ 0, & \text{otherwise.} \end{cases} \tag{24}$$

In the numerator of Eq. (23) the total skill requirement for skill $j$ is calculated in time units. The denominator consists of the scarceness of skill $j$ multiplied by the sum of the predefined durations of all the activities. For skills that are required by a considerable amount of activities (high $r_{ij}$ in the numerator) the skill criticality will be higher. Similarly, skills that are not well-represented in the workforce pool (low $y_{jk}$ in the denominator) will be very critical. The skill criticality will be utilised during the resource assignment in the solution approach and will be used to control the choice of local searches.

**Resource criticality:** The *resource criticality* $RC_k$ determines the weight of each resource $k$ in the project. The concept of resource criticalities is also retrieved from Bellenguez-Morineau and Néron (2007), but is approached differently in this study. Two types of resource criticality are presented. The *basic resource criticality* $RC_k^b$ incorporates both the breadth and depth for each resource $k$ into one formula.

$$RC_k^b = b_k + \sum_j ((1 - d_{jk}) \cdot y_{jk}) \tag{25}$$

The higher the breadth of a resource, the higher its priority will be. Consequently, the most flexible resources are deemed the most important. The depth differentiates resources with equal breadth and hence serves as a tie-breaker. The $RC_k^b$ only depends on the workforce characteristics and is not affected by the requirements of the activities.

In most cases, the weight of a resource depends not only on its skills, but also on the skill requirements of the project. Hence, the *advanced resource criticality* $RC_k^a$ integrates for each resource $k$ both the breadth, the depth and the project-specific ranked skill criticality $RSC_j$. The $RSC_j$ ranks the skills from low to high based on their skill criticality and gives them the value of their position in this list. The $RSC_j$ of the most critical skill is $|S|$, which is the total number of skills in the project. The $RSC_j$ of the least critical skill is equal to 1. The advanced resource criticality $RC_k^a$ of a resource $k$ is calculated as follows:

$$RC_k^a = \sum_j ((RSC_j - d_{jk} + 1) \cdot y_{jk}) \tag{26}$$

In this case, the resources that master the most critical skills are considered the most important. Similar to the basic resource criticality, the depth is utilised as a tie-breaker. The $RC_k^a$ is deemed more advanced because it relies on the skill requirements as well as the resource characteristics. The resource criticality will be used in the algorithm to assign resources to activities and is a key concept in one of the local searches.

**Activity criticality:** The well-known *activity criticality* $AC_i$ determines whether activity $i$ is part of the critical chain $CC$ or not. The critical activities have zero slack, with $slack_i = 0$ for activity $i$. This concept is crucial to this approach because critical and non-critical activities will be handled differently in the local searches.

### 4.2. Genetic algorithm

Since the MSRCPSP is an NP-hard problem (Bellenguez & Néron, 2004), we solve it with a genetic algorithm, which belongs to the class of evolutionary algorithms. GAs have successfully been applied to the RCPSP and several other project scheduling problems (Hartmann & Briskorn, 2010). Fig. 4 presents a graphical overview of our GA implementation, the different components will be explained in the following sections.

#### 4.2.1. Representation

Since the MSRCPSP consists of a scheduling and assignment problem, the representation is also divided into two separate lists.

The *activity list* (AL) is used to represent a solution to the scheduling problem. In an activity list, every activity occurs once and is displayed in a precedence feasible manner, this means that the predecessors of an activity appear to the left of that activity in the list. The order of the activity list shows the order in which the activities will be scheduled depending on their eligibility. Activities are eligible for scheduling if their predecessors have been scheduled and if there are enough available resources to fulfil their skill requirements. Hartmann (1998) pointed out that an AL is an efficient technique to represent a schedule.

To represent solutions to the assignment problem, we introduce the *priority rule list* (PL). In this list of length $|N|$, every activity is appointed a value that corresponds to a priority rule, which consists of $|R|$ values. If activity $i$ is scheduled, the algorithm will assign the resources according to value $PL_i$. For each activity $i$ and for

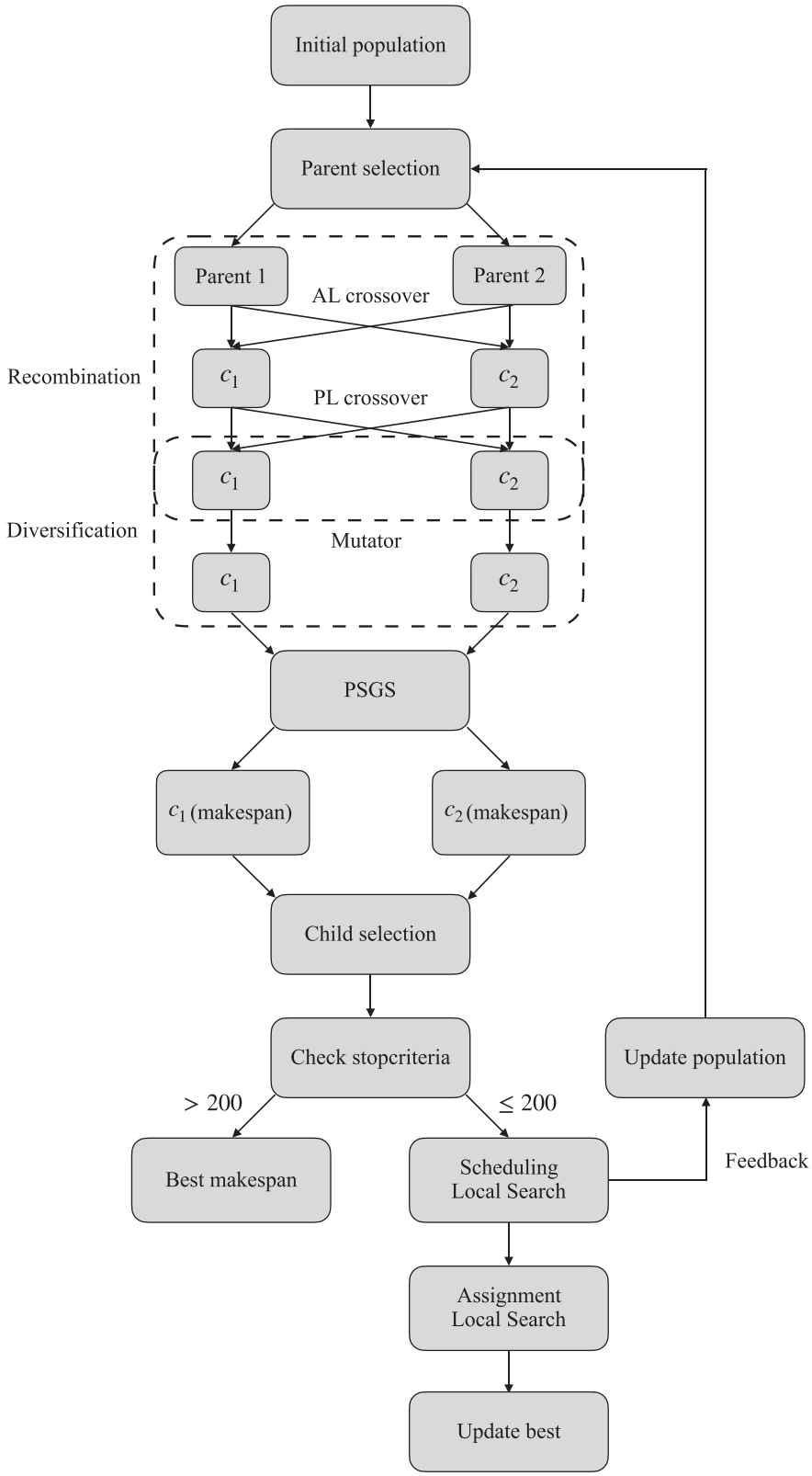


**Fig. 4.** An overview of the genetic algorithm.

each skill $j$, the solution approach goes through the values of $PL_i$, assigns the first available resource that masters skill $j$ to the activity, and iterates through the list until $r_{ij}$ resources are assigned. The set of resource priority rules, that is assembled to enable the assignment of resources to activities in this solution approach, is explained in Section 4.2.2. Hartmann (1998) presented a similar list to apply priority rules to schedule activities.

An alternative to the PL is the resource list (RL), which was proposed by Almeida et al. (2018a) in a multi-skilled environment. The resource list is a single list of the resources in the project and has

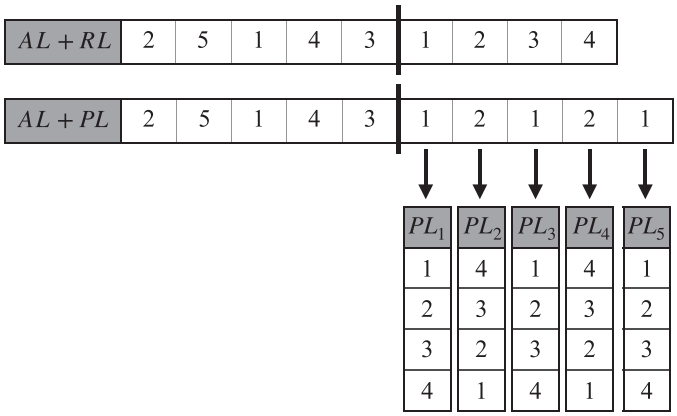


**Fig. 5.** Comparison of alternative solution representations ($|N| = 5$, $|R| = 4$).

**Table 4**
Classification of priority rules.

| Breadth | Depth | Resource criticality |
|---|---|---|
| Lowest breadth first | Highest average depth first | Lowest basic resource criticality first |
| Highest breadth first | Lowest average depth first | Lowest advanced resource criticality first |
| Lowest grouped breadth first | Lowest total negative depth first | Highest advanced resource criticality first |

length $|R|$. It displays the priority with which the resources are assigned to activities and remains the same for all the project activities. If an activity is scheduled, the algorithm will assign resources in the order of the RL.

An advantage of the PL is that each activity can have a different priority rule, which means more resource assignments can be represented by this solution vector in comparison to the RL. However, a disadvantage of the priority rule list is that the priority rules need to be created before the start of the project, while the resource list is generated according to the project's requirements and the generated project schedules in the genetic algorithm.

In Fig. 5, we compare a RL and a PL. In this example, when the RL is used, the algorithm checks for each skill if resource 1 can be assigned to the activity, then the algorithm goes to resource 2 and continues until the skill requirements are satisfied. In the PL, the algorithm assigns resources to activity 1 using priority rule 1, which considers for each skill resource 1 first, followed by resource 2 and so on. Resources are assigned to activity 2 with a different priority rule $PL_2 = 2$. The priority rules are shown in the vertical lists in Fig. 5 and explained in Section 4.2.2.

### 4.2.2. Resource priority rules

This section explains the 9 priority rules that will be used by the PL discussed in the previous section. Some of them are based on the breadth of the resources or on the depth of their skills and others are based on the basic or advanced resource criticality (Table 4).

**Lowest Breadth First (LB):** This priority rule assigns resources based on the breadth $b_k$. The resources are ranked in increasing order of the number of skills they master. During the assignment, the single-skilled resources $R_s$ (lowest breadth) are taken into consideration first. Such an approach keeps the most flexible, multi skilled resources $R_m$ available as long as possible. Therefore, this rule tries to optimise the number of parallel scheduled activities, which increases the resource usage and improves assignment effectivity.

**Highest Breadth First (HB):** This rule is the reverse of the previous priority rule. In this case, the resources who master the most skills will be assigned first. Computational experiments on a testset showed that on average better solutions were found for diversified sets of priority rules. Therefore, the goal of this rule is to introduce controlled diversity into the set of resource priority rules.

**Lowest Grouped Breadth First (LGB):** The resources in this priority rule are sorted dependent on their breadth after being categorised based on the skill criticality of the skills they master. The goal of this rule is to keep the most critical skills available. Resources that master the second most critical skill but do not master the most critical skill will be given priority next, etc.

**Highest Average Depth First (HAD):** This rule uses depth $d_{jk}$ to assign the multi-skilled resources. The resources are ordered based on the average depth of their mastered skills. Workers with the highest average depth will be given the highest priority. The aim of this rule is to assign the most efficient resources first, which will lower the duration of the individual activities, and can therefore lower the overall makespan.

**Lowest Average Depth First (LAD):** This priority rule is the opposite of the previous rule. Resources will be ranked from the lowest to the highest average depth, which indicates that the least efficient resources will be prioritised. Similarly to Highest Breadth First, this rule adds diversity to the set of priority rules.

**Lowest Total Negative Depth First (LTND):** The total negative depth is the basis for this resource priority rule. The negative depth $d^-_{jk}$ of resource $k$ for skill $j$ is the depth that is below the standard depth level of 1.

$$d^-_{jk} = \begin{cases} 1 - d_{jk}, & \text{if } d_{jk} < 1. \\ 0, & \text{if } d_{jk} \geq 1 \text{ or } d_{jk} = 0. \end{cases} \tag{27}$$

Resources who are efficient at every skill they master (low total negative depth) will get the highest priority. Hence, it is less likely that the adjusted duration for activities will exceed their predefined duration $p_i$ in this case.

**Lowest Basic Resource Criticality First (LBRC):** This rule considers the basic resource criticality $RC^b_k$ for each resource $k$ as explained in Section 4.1. In this case, the least flexible resources (lowest breadth) are prioritised and the depth is used as a tie-breaker. The goal of this rule is to keep the most flexible resources available, while also focussing on assigning the most efficient resources.

**Lowest Advanced Resource Criticality First (LARC):** The advanced resource criticality $RC^a_k$, which is also explained in Section 4.1, is the basis for the next priority rule. In this case, the resources with the lowest $RC^a_k$ are considered first to assign to the activity. The aim of this rule is to minimise the makespan through effective assignment.

**Highest Advanced Resource Criticality First (HARC):** This rule is the opposite of the previous rule. It assigns the resources with the highest advanced resource criticality first. This rule also introduces controlled diversity and facilitates the search for diverse resource assignments.

### 4.2.3. Initial population

To initialise the genetic algorithm an initial population is generated. In this research, the initial activity lists are randomly generated except for two lists. In one list, the activities are sorted in decreasing order of the sum of the criticality of the skills that they require. The other activity list sorts the activities in descending order of the skill requirements. The priority rule lists are always randomly generated. The two best solutions in the population, based on the makespan, are added to the set of elite solutions.

### 4.2.4. Parent selection and crossovers

Two parent solutions are selected from the population. Every solution in the population is selected once as the first parent. The second parent is chosen randomly using a 2-tournament selection

procedure. A crossover is implemented to generate two child solutions $c_1$ and $c_2$ by recombination of the two parents. In our procedure, two different crossovers are implemented. The *low demand range crossover* modifies the AL and is a two-point crossover that creates new solutions based on the total resource requirement of an activity. The total resource requirement $q_i$ of an activity $i$ is defined as the sum of all required skills $r_{ij}$, multiplied by the standard duration of the activity $p_i$.

$$q_i = p_i \cdot \sum_j r_{ij} \tag{28}$$

This crossover utilises ranges of activities to find the crossover points. A range is defined as a subset of adjacent activities of an AL. The low demand range crossover looks for ranges of activities in the AL of parent 1 that have a lower total resource requirement $q_i$ than the average total resource requirement $\overline{q}$. From the set of activities with a $q_i$ lower than $\overline{q}$, the two longest ranges are selected. The middle points of these two ranges will be the two points of this two-point crossover. For ranges with an odd length the middle point is set to the right of the middle activity. To create a new solution $c$, the activities outside the two-points are copied from the first parent and the activities between these two points are inserted using the order of the activity list of the second parent. The second crossover is a *random two-point crossover*, which operates on the PL or RL. This crossover has been used for the MSRCPSP in Valls et al. (2009). This crossover randomly selects two points and then creates a new child solution similarly to the crossover presented above. The child solution $c$, is created by copying the activities outside the two crossover points from parent 1, and inserting the activities between the two points using the order of parent 2.

#### 4.2.5. Mutators

The aim of the mutator is to generate new activity sequences or resource assignments, different from the modifications produced by the two crossovers. Two mutators are constructed and implemented in the genetic algorithm. The first mutator is a swap-mutator, which swaps two random values in either the AL or the PL or the RL of a child solution. Secondly, the modify-mutator, which operates on the priority rule list, modifies a random value in the list.

#### 4.2.6. Schedule generation scheme and update procedure

Schedules of the generated child solutions are constructed using the parallel schedule generation scheme (PSGS), presented in Bedworth and Bailey (1982). The PSGS for the MSRCPSP works similar to the PSGS for the RCPSP. The PSGS consists of at most $|N|$ stages and applies a time-incrementation, which means that the time $t$ is incremented between each of $|N|$ stages. Each stage is a decision point at time $t$ with a completed set, an active set and a decision set of activities. At each stage, the active, or ongoing, activities with a finish time equal to $t$ are moved to the completed set. An activity $i$ is selected from the decision set, which is the set of eligible activities for scheduling, based on the order of the AL. Then the algorithm assigns resources to this activity according to the PL or the RL. If the following two constraints are satisfied, activity $i$ is scheduled, with $s_i = t$, and is moved to the active set.

- Resource constraint: the number of available resources at time $t$ is higher than or equal to the number of required resources by activity $i$. This constraint is identical to the renewable resource constraint of the RCPSP.
- Skill constraint: the mastered skills of the available resources at time $t$ must satisfy the number of required skills $r_{ij}$ by activity $i$ for each skill $j$.

If at least one of the constraints is not satisfied, we move on to the next activity in the AL that is part of the decision set. This step is performed until the decision set of activities is empty, after which the time is incremented and the procedure proceeds to the next decision point. The method terminates when all activities are in the completed set.

Then the best child, based on the makespan, is selected and replaces parent 1 in the population. After updating each solution in the population with its child solution, the procedure checks if the best solutions in the population can replace the two solutions in the elite set.

#### 4.2.7. Local searches

In this section, we propose three local searches which are applied to the schedules of the solution vectors in the population.

*Scheduling local search*

This local search is based on the well-known iterative scheduling algorithm proposed by Li and Willis (1992). This local search has been used for the MRCPSP by Van Peteghem and Vanhoucke (2010). This scheduling technique modifies the start- and finish times of the activities, while keeping the assignments fixed. More specifically, it applies forward and backward scheduling to generate left- and right-justified schedules, which means scheduling the activities, respectively, as soon as possible and as late as possible. Hence, it is able to create opportunities for left and right shifts of activities in order to improve the makespan.

*Assignment local searches*

This section presents two new local searches that aim at reassigning the resources of a given schedule depending on the maximum value of the skill criticality $SC_j$ for each skill $j$ of the project instance. The skill criticality is used as a selection criterion since it measures the availability of each skill $j$ to the number of times it is required by the activities. The higher the value of $SC_j$, the harder it is to assign resources to activities that require that skill. More precisely, when the maximum value for the skill criticality $SC_j$ exceeds a certain predefined threshold $\tau$, then the algorithm selects the so-called *resource-based local search*, while otherwise, the so-called *activity-based local search* will be applied.

Both local searches make use of a *resource reassignment procedure* - abbreviated as `ReAssign()` in the pseudocode of Algorithms 1 and 2 – that modifies the current resource assignment to activities. To that purpose, for a given schedule, the set of resources $R$ is subdivided into 3 separate subsets $A_i^r$, $U_i^r$ and $O_i^r$ for each activity $i$ as follows:

**Algorithm 1** Resource-based local search.

```
for i ∈ AL_1,...,AL_|N| do
    Calculate U_i^r
    U_i^r ← U_i^r ∪ A_i^r
    A_i^r = ∅
    ReAssign(i, U_i^r, {LARC, Highest Depth})
    Update p_i^a ⟹ Update f_i
    if f'_i ≥ f_i then
        ∀i' ∈ N|s_i' > f_i : s_i' ← s_i' + (f'_i − f_i)
    End if
    for i' ∈ 1, ..., |N| : s_i' ≥ f'_i do
        if LeftShift(i') then
            Decrease s_i' and f_i'
        End if
        if ReAssign(i', U_i^r, LARC) then
            s_i' ← s_i.
            Update p_i'^a ⟹ Update f_i'
            Update U_i^r
        End if
```

**Algorithm 2** Activity-based local search.

```
for i ∈ AL_1,...,AL_|N| do
    if i ∈ CC then
        Calculate U_i^r
        U_i^r ← U_i^r ∪ A_i^r
        A_i^r = ∅
        ReAssign(i, U_i^r, Highest Depth)
        Decrease p_i^a ⟹ Decrease f_i
        for i' ∈ 1, ..., |N| : s_i' ≥ f_i' do
            if LeftShift(i') then
                Decrease s_i' and f_i'
            End if
    else if i ∉ CC then
        Calculate U_i^r
        U_i^r ← U_i^r ∪ A_i^r
        A_i^r = ∅
        ReAssign(i, U_i^r, Lowest depth)
        Increase p_i^a ⟹ Increase f_i
        if ResourceConflict() then
            Undo ReAssign()
        End if
    End if
```

$A_i^r$: This set contains all resources assigned to activity $i$.

$$A_i^r = \{k \in R \mid \textstyle\sum_{j \in S} x_{ijk} = 1\} \tag{29}$$

$U_i^r$: This set contains all unassigned resources in the interval $[s_i, f_i[$, i.e. all resources not assigned to any activity $i \in N$ at the time interval $t = s_i, \ldots, f_i - 1$.

$$U_i^r = \{k \in R \mid \textstyle\sum_{i' \in S(i)} \sum_{j \in S} x_{i'jk} = 0\} \tag{30}$$

$O_i^r$: This set contains all other resources that are assigned to any activity $j \in N \setminus i$ at at least one time instant $t \in [s_i, f_i[$.

$$O_i^r = \{k \in R \mid \textstyle\sum_{i' \in S(i) \setminus i} \sum_{j \in S} x_{i'jk} > 0\} \tag{31}$$

with $S(i) = \{i' \in N \mid s_i < f_{i'} \wedge s_{i'} < f_i\}$ and obviously $R = A_i^r \cup U_i^r \cup O_i^r$.

The procedure ReAssign($i, U_i^r$,PR) makes use of these three sets for each activity $i$ and simply removes all resources from the set $A_i^r$, puts them in $U_i^r$ and then creates a new resource assignment for activity $i$ using the resources in set $U_i^r$ by applying a resource priority rule PR. The choice of the priority rule depends on which local search is called in the procedure.

*Resource-based local search (high skill criticality)*

The resource-based local search is used when the maximum value of $SC_j$ exceeds the threshold $\tau$. A high value for $SC_j$ implies that the project instance has skills that are either required by a lot of activities, or that only a few resources master this specific skill. Hence, assigning resources to activities that require skills with high skill criticality values should be performed with care, since a poor assignment for an activity $i$ will prevent other activities $i'$ (requiring the same skills) to be scheduled in parallel with activity $i$. The local search therefore reassigns resources for activities in order to improve the schedule, the pseudocode is presented in Algorithm 1.

The local search enumerates all activities $i \in \{1, \ldots, |N|\}$ in the order of the activity list (i.e. from $AL_1$ to $AL_{|N|}$) and then aims at improving the quality of the resource assignments in two phases that will be explained along the following lines.

**Phase 1.** In a first phase, the local search calls the previously mentioned ReAssign() procedure to try to reassign the currently assigned resources for activity $i$. As said, this procedure removes all resources from activity $i$ ($A_i^r = \varnothing$), adds them to set $U_i^r$, and then assigns new resources from $U_i^r$ to $A_i^r$ using either the LARC rule of Section 4.2.2 or assigns the resources with the highest depth, with an equal chance. This rule can assign the resources with the lowest values for the $RC_k^a$ first to the activity, which means that the resources with high $RC_k^a$ values will remain in the set $U_i^r$. Consequently, a ReAssign() results in an increase in the total resource criticality (i.e. $\sum_{k \in R} RC_k^a$) for set $U_i^r$ and a decrease in set $A_i^r$. Indeed, since resources with high $RC_k^a$ values typically master skills with high $SC_j$ values, the set of unassigned resources $U_i^r$ now consists of more critical resources that are available to other activities that can be scheduled in parallel with activity $i$ (which will be checked in the second phase of the local search).

After such a ReAssign(), activity $i$ has received a new set of resources, and the adjusted processing time $p_i^a$ must therefore be updated according to the formulas of Section 3. This results in a new finishing time $f_i'$ for activity $i$ which can be smaller than, equal to or larger than its original finishing time $f_i$. Obviously, such change in the finishing time of activity $i$ has an impact on the other activities starting later in the schedule, and therefore the schedule requires an update. However, the schedule will be updated only when the finishing time of activity has increased ($f_i' > f_i$). In this case, each activity $i'$, scheduled after activity $i$ ($s_{i'} \geq f_i$), is right shifted by increasing the start $s_{i'}$ and finish times $f_{i'}$ with $f_i' - f_i$. In case $f_i' \leq f_i$, no schedule update is performed. The algorithm has now a new set of unassigned resources $U_i^r$ and continues with the second phase of the local search.

**Phase 2.** In the second phase, the local search enumerates all activities $i'$ with a start time $s_{i'} \geq f_i'$ in increasing order of start time and tries to schedule them earlier in the schedule. First, the algorithm tries to left shift (LeftShift()) activity $i'$ as much as possible by checking whether activity $i'$ can be scheduled between $f_i'$ and $s_{i'}$ without changing its resource assignments. Afterwards, the algorithm continues by checking whether activity $i'$ can be scheduled at start time $s_i$ by reassigning its resources. Such check is done using the same ReAssign() as explained before. If a ReAssign() is possible, activity $i'$ is scheduled at start time $s_{i'} = s_i$ and $U_i^r$ is updated (i.e. the resources newly assigned to activity $i'$ are removed from the set).

The LeftShift() and ReAssign() procedures of phase 2 are called for each activity $i'$ until all activities in the remaining schedule are considered, or until the set $U_i^r$ is empty, after which the algorithm selects activity $AL_{i+1}$ and starts again with phase 1. Both phases are repeated until the last activity $AL_n$ is considered. This local search might result in a new improved schedule thanks to a better assignment of resources to activities.

*Activity-based local search (low skill criticality)*

The activity-based local search is applied when all $SC_j$ values are lower than the threshold $\tau$. Low $SC_j$ values imply that all skills are well-presented in the workforce. Due to these higher skill availabilities, it will be less difficult to assign resources to the required skills for activities. Therefore, the activity-based local search will focus less on reassigning resources using the resource criticality $RC_k^a$ as in the resource-based local search, but instead will aim at reassigning resources based on their depth $d_{jk}$ to optimise the makespan. The activity-based local search works in two simultaneous phases. The first phase aims at reducing the project makespan by reassigning resources to activities on the critical chain (further abbreviated as set $CC$), while the second phase aims at assigning less efficient resources to activities not on the critical chain (i.e. activities $i \notin CC$).

**Phase 1.** The first phase only deals with the activities $i$ on the critical chain from $AL_1$ to $AL_{|N|}$ and updates the sets $U_i^r$ and $A_i^r$ similarly to the resource-based local search. More specifically, the resources assigned to activity $i$ are removed ($A_i^r = \varnothing$) and added to set $U_i^r$, and the ReAssign() is called to improve the current resource assignment. The assignment is done by assigning resources

with the highest depth first. Since the newly assigned resources of activity $i$ ($A_i^r$) now have a higher (or equal) depth value, the adjusted processing time $p_i^a$ must be updated and will be lower than (or equal to) the processing time before the reassignment. Consequently, the new finishing time $f_i'$ must be updated too ($f_i' \leq f_i$), and all activities $i'$ that start later than the new finishing $f_i'$ must be shifted forward using the `LeftShift()` procedure discussed earlier. If necessary, the procedure updates the set of the activities on the critical chain, and continues with the enumeration of Algorithm 1.

**Phase 2.** The second phase tries to increase the durations of non-critical activities $i \notin CC$ without changing the start times of any other activity in the schedule in order to effectively use the less efficient resources. In a similar way as Phase 1, the procedure removes all resources from activity $i$ and adds them to $U_i^r$, followed by a possible resource reassignment using the `ReAssign()` procedure. However, this time the resources with the lowest depth values are assigned first. Such a reassignment works exactly the opposite way as in Phase 1, and leads to an increase in the total depth of set $U_i^r$ and a decrease for set $A_i^r$. As always, the adjusted processing time $p_i^a$ must be updated which will now - due to the lower average depth values - be higher than the processing time before the reassignment. If the increase in the $p_i^a$ does not affect the start times of any other activity, and no new resource conflict is generated (i.e. the procedure `ResourceConflict()` returns FALSE), then the reassignment is accepted and the finishing time of activity $i$ is increased. Otherwise, when the new adjusted processing time $p_i^a$ causes a shift of any other activity due to a new resource conflict (i.e. `ResourceConflict()`) returns TRUE), the resource reassignment for activity $i$ is reset and restored to the initial assignment. Consequently, any reassignment that will lead to an increased processing time with no impact on the other activity start times and renewable resource constraints will increase the duration of the non-critical activity without affecting the total makespan of the project.

## 5. Data generation

In this section, we present a new generated dataset for the MSRCPSP under study. The data will be generated using predefined project parameters that characterize the problem (Section 5.1). These parameters will be set on different values such that a dataset consisting of 6,250 projects can be generated (Section 5.2).

### 5.1. Input parameters

In order to generate instances for the MSRCPSP, project networks with renewable resources and available skills must be generated. The project instance generator of Almeida et al. (2016) can be used in which the number of resources $|R|$ is one of the input parameters, after which the required skills are then randomly assigned to the resources. We could use this generator, but it would not allow us to investigate the impact of the workforce size and the skill availabilities on the project objectives in a controlled way. Therefore, we have decided to propose a new data generator and corresponding dataset that generates project instances under a controlled design. More specifically, the project networks will be generated under a controlled network structure, while the full spectrum for skill and resource availabilities will be incorporated based on three resource parameters.

**Network structure:** The networks will be generated by the *RanGen2* generator proposed by Vanhoucke, Coelho, Debels, Maenhout, and Tavares (2008). This network generator generates activity-on-the-node networks with a given value for the *serial/parallel* network indicator (SP) that varies between 0 (completely parallel networks) to 1 (completely serial networks) and has been used in various studies in the academic literature.

Once the activities of the project are generated and put in a project network, the skill requirements for each activity, as well as the multi-skilled resources with breadth and depth values must be generated. This generation will be done using three skill parameters which will be discussed along the following lines.

**Skill factor:** The skill factor of activity $i$ is abbreviated as $SF_i$ and has been introduced by Correia et al. (2012) as follows:

$$SF_i = \frac{\sum_j z_{ij}}{|S|} \tag{32}$$

with the binary variable $z_{ij}$ equal to 1 if the requirements $r_{ij}$ of activity $i$ for skill $j$ are greater than 1 and 0 otherwise.

$$z_{ij} = \begin{cases} 1, & \text{if } r_{ij} > 0. \\ 0, & \text{otherwise.} \end{cases} \tag{33}$$

Eq. (32) shows that the skill factor represents the number of skills requested by an activity in the network. Consequently, the skill factor is a measure for the density of the skill requirement matrix.

**Skill strength:** The strength of a skill $j \in S$ is used to measure the scarceness of the skill, and compares the number of available skills in the workforce with the requirements of these skills by the activities in the project. The Skill Strength $SS_j$ for skills $j$ is similar to the Resource Strength $RS_k$ for each resource $k$ for multi-mode projects as defined by Kolisch, Sprecher, and Drexl (1995), and can be calculated as follows:

$$SS_j = \frac{a_j - a_j^{min}}{\sum_i r_{ij} - a_j^{min}} \tag{34}$$

The term $\sum_i r_{ij}$ measures for each skill $j$ the total number of units required by all activities. The minimal skill availability $a_j^{min}$ of skill $j$ is equal to the maximum number of units of skill $j$ that is required by a single activity $i$ in the project, calculated as:

$$a_j^{min} = \max_i(r_{ij}) \tag{35}$$

When the availability $a_j$ of skill $j$ is much lower than the total requirement $\sum_i r_{ij}$, the problem might be hard to solve since skill $j$ is very scarce, and $SS_j$ will be relatively close to 0. When the availability $a_j$ is similar to the total requirement, then $SS_j$ lies close to 1, and the problem might be less difficult to solve since skill $j$ is widely available. Obviously, if the availability $a_j$ is lower than $a_j^{min}$, then $SS_j < 0$ and the workforce does not possess enough resources with the right skills to satisfy the requirements of at least 1 activity. If $\sum_i r_{ij} = a_j^{min}$, then the $SS_j$ is assumed to be equal to 1.

**Skill availability:** Since the size of the workforce will have a significant impact on the final project schedule, the *Skill Availability SA* is defined as a measure of how the available skills are distributed over the existing resources and is therefore a measure of the resource scarceness in the problem instance. The *SA* describes the average number of skills the resources master and measures the flexibility of the multi-skilled resources. It aims at making a distinction between having a very multi-skilled workforce (low *SA*) and a predominantly single-skilled workforce (high *SA*).

$$SA = \frac{|R| - R^{min}}{R^{max} - R^{min}} \tag{36}$$

The minimum number of resources $R^{min}$ necessary for the project can be determined by solving a small assignment problem based on the available number of skills $a_j$. The $R^{max}$ is the maximum number of resources over which the available skills can be distributed taking the minimal breadth $b^{min}$ value of a resource into account $R^{max} = \frac{\sum_j a_j}{b^{min}}$. If $R^{max} = R^{min}$, then the *SA* is assumed to be equal to 1.

**Table 5**
Parameter settings for dataset and test set.

| Parameter | Dataset |
|---|---|
| $N$ | 30 |
| $S$ | 4 |
| $R$ | varying |
| $SF_i$ | 0.25 to 1 (steps of 0.25) with $\frac{\alpha}{\alpha_{max}}$: 0 to 1 (steps of 0.25) |
| $SS_j$ | 0 to 1 (steps of 0.25) |
| $SA$ | 0 to 1 (steps of 0.25) |
| $b^{min} - b^{max}$ | 1 to 4 |
| $d^{min} - d^{max}$ | 0.5 to 1.5 (steps of 0.25) |
| max. resources required per activity | 12 |
| max. resources required per skill in an activity | 4 |
| SP-values | 0.1 to 0.9 (steps of 0.2) |
| Total instances | 6250 |

### 5.2. New dataset

Based on the parameters described in the previous section, a new dataset is presented for which the values for each parameter are set in Table 5. The instances of the dataset are created by *RanGen2* and are 50 projects from the RG30 dataset (Vanhoucke et al., 2008). Each instance consists of a network with 30 activities and the structure of the network is measured by the SP indicator that ranges from 0.1 to 0.9 in steps of 0.2. For each of these 5 settings, 10 different project networks were selected. The number of skills $|S|$ is equal to 4 and the number of resources $|R|$ depends on the values of the resource parameters.

The average value for the $SF_i$ over all activities is set to 4 different values, but the number of requested skills can still vary among activities. To that purpose, the variability in the values for $SF_i$ over the different activities $i$ is generated using the $\frac{\alpha}{\alpha_{max}}$ metric of Labro and Vanhoucke (2008) which varies from 0 to 1 in steps of 0.25. When $\frac{\alpha}{\alpha_{max}} = 0$, the $SF_i$ is the same for all project activities and equal to the average $SF_i$ value. For $\frac{\alpha}{\alpha_{max}} = 1$, the $SF_i$ vary between the different activities as much as possible. Furthermore, both the skill strength $SS_j$ and the skill availability $SA$ vary from 0 to 1 in steps of 0.25 to analyse their effects. Other parameters are set to fixed values as follows, the minimum and maximum breadth, $b^{min}$ and $b^{max}$, are respectively, set to 1 and 4. Similarly, the depth also has a minimum and maximum value, $d^{min}$ and $d^{max}$, which are set to respectively 0.5 and 1.5. The depth $d_{jk}$ can take on every value between $d^{min}$ and $d^{max}$ in steps of 0.25. The last two fixed parameters set the maximum number of resources that can be required by an activity, which is equal to 12, and the maximum number of resources that can be required by a single skill in an activity, which is equal to 4. In total, this creates a complete dataset of $5 \times 5 \times 5 \times 5 \times 10 = 6250$ instances.

## 6. Computational experiments

Computational experiments have been executed to compare the performance of the approach explained in Section 4 with the performance of some variant approaches for benchmarking purposes. Section 6.1 briefly explains the parameter fine-tuning. The experimental results of the different approaches and a comparison of the new approach to a benchmark is discussed in Section 6.2. A thorough analysis of the resource priority rules and their impact on the results is presented in Section 6.3. In Section 6.4 we provide an analysis of the impact of the resource parameters. The solution approaches are coded in C++ and are tested on an Intel Xeon Gold 6140 processor with 2.3 gigahertz.

**Table 6**
Performance of 10 solution approaches (in % relative improvement gap to approach 1).

| Approach | Scheduling | Assignment | LS | Makespan |
|---|---|---|---|---|
| 1 | Random | Random | No | / |
| 2 | Random | Random | Yes | 2.78% |
| 3 | Random | PL | No | 12.45% |
| 4 | Random | PL | Yes | 26.96% |
| 5 | AL | PL | No | 27.48% |
| 6 | AL | PL | Yes | 32.42% |
| 7 | AL | RL | No | 26.49% |
| 8 | AL | RL | Yes | 31.23% |
| 9 | AL | PL+RL | Yes | 33.02% |
| 10 | AL | PL(RL only) | Yes | 31.67% |

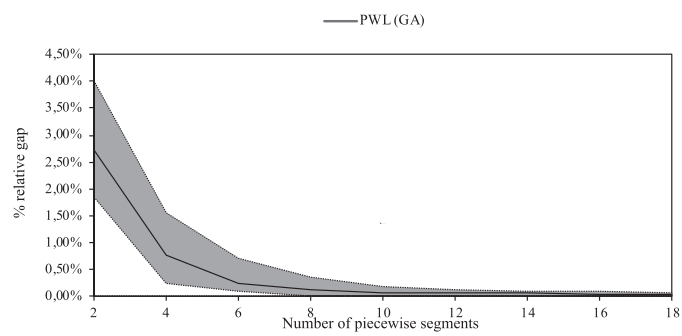


**Fig. 6.** Performance of piecewise linear calculation of adjusted processing time (in % relative gap to continuous calculation).

### 6.1. Parameter fine-tuning

A small subset of 81 instances, that is representative for the whole dataset, was used to fine-tune the parameters of the algorithm. The resulting parameters are as follows: a population size of 75 solution vectors, a crossover rate of 0.8 and a mutation rate of 0.2. Furthermore, the algorithms are truncated after 15,000 schedules are generated.

### 6.2. Algorithmic design

This section evaluates the performance of the genetic algorithm (GA) under 10 different settings. Table 6 provides a summary of 10 solution approaches and shows the different components used for solving the scheduling and assignment problem.

In all computational experiments of Section 6 of this manuscript, we used the non-linear adjusted processing times as shown in Eq. (2) for the genetic algorithm, the linearised adjusted processing times of Eqs. (15)–(22) are only used by the CPLEX solver. This CPLEX model makes use of 10 segments for the linearisation based on computational experience shown in Fig. 6. This figure shows the quality of the genetic algorithm when the non-linear function is replaced by the linearisation for a different number of segments ranging from 2 to 18. The *Y*-axis of this figure shows the increase in the project makespan due to the linearisation relative to the best found makespan with non-linear adjusted processing times. These differences are shown as 95% intervals, with the black line the average values. The figure shows that the differences are marginal for 10 segments (only 0.06%). Consequently, using 10 segments for the CPLEX procedure will not lead to huge increases in the project makespan due to the linearisation.

The 10 algorithmic implementations differ in how the scheduling and assignment problems are solved. The *scheduling problem* requires the creation of an activity list (AL) to generate resource-feasible schedules (approaches 5–10) and will be compared with a naive method that randomly selects activities in the schedule gen-

**Table 7**
Analysis of the priority rule set composition (in % relative improvement gap to approach 1).

| Approach | Scheduling | Assignment | # of added rules | Makespan |
|---|---|---|---|---|
| 9 | AL | PL+RL | 1 | 32.87% |
| 9 | AL | PL+RL | 5 | 33.02% |
| 9 | AL | PL+RL | 10 | 33.00% |
| 9 | AL | PL+RL | 20 | 32.85% |
| 10 | AL | PL(RL only) | 1 | 31.19% |
| 10 | AL | PL(RL only) | 5 | 31.54% |
| 10 | AL | PL(RL only) | 10 | 31.67% |
| 10 | AL | PL(RL only) | 20 | 31.65% |

eration scheme (approaches 1–4). The *resource assignment problem* can also rely on randomly generated resource assignments (approaches 1 and 2), or can be solved using the two representations discussed in Section 4.2.1. Each version of the problem can be solved with or without the use of the local search of Section 4.2.7. Consequently, the combination of random solutions for both problems, the use of activity lists for the scheduling problem, the use of priority lists or resource lists for the resource assignment problem and the incorporation of the local search or not results in 8 different algorithmic implementations.

Approach 9 and 10 are somewhat special and require some extra explanation. Solution approaches 9 and 10 make use of the previous solution approaches by extending the current number of priority rules (Section 4.2.2) to more rules in two different ways. In approach 9, we extend the list of 9 priority rules by first running approach 8 under a strict stop criterion, and saving the best found solutions. These solutions have all been constructed using a resource list (RL) to assign resources. Such resource list can then act as a priority rule and be added to the current set of 9 priority rules. Consequently, approach 9 is similar to approach 6 but now contains a larger set of priority rules found by running approach 8 first and then selecting the resource lists that resulted in the best possible assignments as new priority rules. In approach 10, a similar logic is followed but now the 9 priority rules of Section 4.2.2 are removed, and replaced by the best found priority rules by running approach 8 first. Approaches 9 and 10 are designed to test (1) whether the current set of 9 priority rules is sufficient to solve the resource assignment problem and (2) whether a (small or large) extension of this set with good-performing new rules can lead to significant improvements. The first four columns of Table 6 show how the 10 approaches are assembled. The results of the experiments are shown in the column "Makespan" and report the percentage improvement compared to Approach 1. The results are shown as average improvements for all 6250 project instances.

The results of Table 6 can be summarised as follows. First, the table shows that any approach with randomness (1–4) performs worse than the other approaches, which is an indication that the components of our algorithm perform better than a random search. Second, the table reveals that the priority list (PL) works better than the resource list (RL) for solving the resource assignment problem. This shows that using 9 priority rules to assign resources outperforms the use of a more traditional resource list that is identical for each activity. The table also shows that a combination of the 9 priority rules with some additional resource lists as proposed in Approach 9 generates even better results, which indicates that the idea of using a resource list should not be completely abandoned. However, Approach 10 shows that only using resource lists – even the ones that have shown to perform well – does not lead to the best solutions.

Table 7 displays additional results for Approaches 9 and 10. More specifically, the table analyses how many resource lists – found by running Approach 8 first – should be added to the set of 9 priority rules. The results show that the number of additional resource lists should not be very large, and varies between 1 and 5 for approach 9 and up to 10 for Approach 10. The results for Approach 9 show that the 9 priority lists already provide good schedules, and there is no need to extend these 9 priority rules with many other features. Indeed, adding many more resource lists (more than 5) results in a resource assignment problem with too many options to choose from, which leads to projects with a higher makespan. For Approach 10, up to 10 resource lists can be added, which is obviously higher than for Approach 9 since it removes the 9 priority rules and completely replaces them with resource lists that are known to work well. However, the results are not as good as Approach 9 which is again a good indication that the 9 priority rules work well.

Furthermore, we compare the solution quality of our approach to solutions found from solving the MILP model with CPLEX 12.6. We solved all instances for four time limits: 30 seconds, 1 minute, 5 minutes and 10 minutes (see Table 8). To analyse the results, we divided the dataset into 25 subsets of 250 instances, based on the amount of resources $|R|$ and the SP-value. Table 8 classifies the solutions into three classes:

- Optimal(%OPT): the percentage of instances for which an optimal solution was found.
- Feasible(%FNO): the percentage of instances for which CPLEX could find a feasible but non-optimal solution. For all these instances, the CPLEX model reached its time limit and provided the best found feasible solution as the output.
- Infeasible: the percentage of instances (100-%OPT-%FNO) for which CPLEX could not find any feasible solution within the time limit (values not shown in table).

Table 8 also shows the average gap percentage (%Gap) of the solutions in %OPT + %FNO:

$$\%GAP = (UB^{CPLEX} - LB^{CPLEX})/UB^{CPLEX} \tag{37}$$

The $LB^{CPLEX}$ is the lower bound calculated by CPLEX through linear relaxation of the integrality constraints, which is the default LB calculation of CPLEX. The $UB^{CPLEX}$ is the best integer solution found by CPLEX, which is the optimal solution for the instances in %OPT and the best feasible solution found for the instances in %FNO. Obviously, the infeasible instances from CPLEX are excluded in this %GAP calculation. From this table it is clear that it is more difficult to find the optimal solution when there is a low amount of resources available in the instance. Additionally, the total number of feasible instances increases with increasing values for the time limit. The last two columns of the table show the results for the genetic algorithm. More precisely, for the instances in %OPT + %FNO, the %GAP is calculated as $((UB^{GA} - LB^{CPLEX})/UB^{GA})$ with $UB^{GA}$ the best found solution of the GA using approach 9 of Table 6. The last column displays the average CPU time (in seconds). The table shows that the %GAP shows the highest values for the instances with low resource availability, but they are nevertheless almost always lower than the %GAP found by CPLEX after 10 minutes. Hence, the GA only needs less than 20 seconds to obtain similar and mostly better results than the MILP method within 10 minutes. This %GAP lowers when the amount of resources increases, similar to the CPLEX model. It is worth noting that the comparison between the GA and MILP models is done only for the %OPT + %FNO instances. Of course, while for the infeasible instances, CPLEX could not find any feasible solution within 10 minutes, the GA could easily find a feasible solution for all instances in CPU times similar to the ones reported.

**Table 8**
Comparison between CPLEX and the solution approach presented in this paper.

| $\|R\|$ | SP | CPLEX (30 seconds) | | | CPLEX (1 minute) | | | CPLEX (5 minute) | | | CPLEX (10 minute) | | | GA (Approach 9) | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | %OPT | %FNO | %Gap | %OPT | %FNO | %Gap | %OPT | %FNO | %Gap | %OPT | %FNO | %Gap | %Gap | CPU time (seconds) |
| | 0.1 | 0.0% | 41.2% | 90.7% | 0.0% | 60.4% | 85.8% | 0.0% | 87.6% | 71.6% | 0.4% | 91.6% | 64.4% | 41,8% | 18.0 |
| | 0.3 | 0.0% | 52.8% | 88.1% | 0.0% | 71.2% | 79.0% | 5.6% | 90.4% | 58.5% | 10.4% | 86.0% | 50.4% | 37,7% | 17.7 |
| Low | 0.5 | 0.0% | 68.0% | 82.3% | 2.8% | 79.6% | 68.3% | 18.0% | 80.0% | 45.1% | 24.8% | 73.2% | 38.3% | 27.8% | 18.1 |
| | 0.7 | 2.0% | 74.4% | 74.3% | 10.4% | 80.8% | 56.5% | 35.6% | 62.0% | 34.2% | 38.8% | 59.6% | 28.7% | 21.5% | 18.6 |
| | 0.9 | 7.6% | 83.2% | 59.2% | 23.6% | 74.4% | 43.1% | 43.6% | 57.4% | 25.5% | 44.4% | 55.6% | 21.9% | 17.6% | 17.1 |
| | 0.1 | 19.6% | 73.4% | 44.2% | 23.2% | 67.6% | 36.3% | 33.6% | 64.0% | 22.7% | 38.0% | 60.8% | 18.5% | 16.7% | 16.9 |
| | 0.3 | 32.0% | 58.0% | 32.4% | 40.4% | 54.8% | 24.9% | 48.8% | 50.8% | 15.6% | 50.4% | 49.2% | 12.8% | 12.5% | 16.0 |
| Low-Med | 0.5 | 44.0% | 51.6% | 21.0% | 50.0% | 48.0% | 15.7% | 57.2% | 42.0% | 9.3% | 57.6% | 42.4% | 7.9% | 7.6% | 15.0 |
| | 0.7 | 54.8% | 43.2% | 13.6% | 59.6% | 40.0% | 10.4% | 62.4% | 37.6% | 5.9% | 63.6% | 36.4% | 5.1% | 5.0% | 15.3 |
| | 0.9 | 60.8% | 39.2% | 9.0% | 66.8% | 33.2% | 6.2% | 70.4% | 29.6% | 4.4% | 70.4% | 29.6% | 3.8% | 3.7% | 15.0 |
| | 0.1 | 53.6% | 41.6% | 14.4% | 57.2% | 40.8% | 9.9% | 62.4% | 37.6% | 5.5% | 62.8% | 37.6% | 4.9% | 5.3% | 15.2 |
| | 0.3 | 70.0% | 27.6% | 8.1% | 72.4% | 26.0% | 6.3% | 75.2% | 24.8% | 3.5% | 76.0% | 24.0% | 3.1% | 3.7% | 15.4 |
| Med | 0.5 | 78.0% | 21.6% | 4.1% | 79.2% | 20.8% | 3.0% | 82.0% | 18.0% | 2.2% | 82.0% | 18.0% | 2.0% | 2.4% | 15.6 |
| | 0.7 | 83.6% | 17.4% | 2.9% | 84.0% | 16.0% | 2.6% | 84.0% | 16.0% | 1.9% | 84.0% | 16.0% | 1.6% | 2.0% | 15.6 |
| | 0.9 | 83.2% | 17.8% | 2.4% | 83.6% | 16.4% | 2.1% | 84.0% | 16.0% | 1.5% | 84.0% | 16.0% | 1.4% | 1.6% | 15.2 |
| | 0.1 | 86.8% | 13.2% | 0.6% | 87.6% | 12.4% | 0.5% | 88.0% | 12.0% | 0.2% | 88.0% | 12.0% | 0.2% | 1.1% | 14.6 |
| | 0.3 | 89.2% | 10.8% | 0.3% | 89.6% | 10.4% | 0.2% | 90.0% | 10.0% | 0.1% | 90.4% | 9.6% | 0.1% | 0.8% | 15.0 |
| Med-High | 0.5 | 93.6% | 6.4% | 0.2% | 93.6% | 6.4% | 0.1% | 93.6% | 6.4% | 0.1% | 93.6% | 6.4% | 0.1% | 0.4% | 15.3 |
| | 0.7 | 96.0% | 4.0% | 0.2% | 96.0% | 4.0% | 0.1% | 96.0% | 4.0% | 0.1% | 96.0% | 4.0% | 0.1% | 0.3% | 15.1 |
| | 0.9 | 96.8% | 3.2% | 0.2% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 0.4% | 14.2 |
| | 0.1 | 93.2% | 6.8% | 0.4% | 93.2% | 6.8% | 0.2% | 93.2% | 6.8% | 0.1% | 93.2% | 6.8% | 0.1% | 0.9% | 12.8 |
| | 0.3 | 98.0% | 2.0% | 0.2% | 98.0% | 2.0% | 0.1% | 98.0% | 2.0% | 0.1% | 98.0% | 2.0% | 0.1% | 0.5% | 13.7 |
| High | 0.5 | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 0.3% | 12.6 |
| | 0.7 | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 96.8% | 3.2% | 0.1% | 0.2% | 13.4 |
| | 0.9 | 92.0% | 8.0% | 0.1% | 92.0% | 8.0% | 0.1% | 92.0% | 8.0% | 0.1% | 92.0% | 8.0% | 0.1% | 0.2% | 12.5 |

**Table 9**
Usage percentage of the resource priority rules in the PL.

| SP | $\|R\|$ | LB | HB | LGB | HAD | LAD | LTND | LBRC | LARC | HARC |
|---|---|---|---|---|---|---|---|---|---|---|
| Low | Low | 10.6% | 8.6% | 7.9% | **16.0%** | 9.3% | **13.1%** | **14.6%** | **13.1%** | 7.4% |
| | Mid | 10.7% | 8.3% | 8.0% | **16.8%** | 8.7% | **13.4%** | **13.8%** | **13.2%** | 7.3% |
| | High | 9.6% | 7.9% | 7.6% | **16.5%** | 8.8% | **15.1%** | **13.5%** | **13.7%** | 7.3% |
| Mid | Low | 7.2% | 6.8% | 6.7% | **29.1%** | 6.6% | 11.2% | 13.5% | 13.2% | 5.9% |
| | Mid | 8.3% | 7.5% | 7.1% | **26.3%** | 7.9% | 11.2% | 12.7% | 12.6% | 6.4% |
| | High | 6.9% | 6.2% | 6.3% | **28.9%** | 6.9% | 11.3% | 13.8% | 13.6% | 6.0% |
| High | Low | 3.3% | 4.5% | 3.9% | **41.9%** | 2.6% | 11.1% | 14.8% | 14.8% | 3.1% |
| | Mid | 3.5% | 4.2% | 4.1% | **41.8%** | 2.7% | 10.9% | 15.2% | 14.8% | 2.9% |
| | High | 4.2% | 4.6% | 3.7% | **41.4%** | 2.9% | 9.5% | 15.3% | 15.4% | 3.0% |

### 6.3. Resource priority rule analysis

In this section, we discuss the usage percentages of the resource priority rules in order to analyse which rules yield good solutions. In Table 9, we present the average fraction each resource priority rule is selected in the best found solutions for different sections of the dataset. To analyse this, the dataset is divided into 9 subsets based on the *SP* and the number of resources $|R|$.

**Low SP:** For lower SP-values, several priority rules have similar usage rates. Since these projects have parallel network structures, the assignment should be as effective as possible such that activities can be scheduled in parallel. There is not one particular rule that can always effectively assign resources, therefore multiple rules are frequently selected.

**Mid SP:** For instances with these SP-values the HAD-rule is prefered, closely followed by LTND, LBRC and LARC. These four rules are selected in more than 50% of the cases. The HAD-rule is used to minimise the durations on the critical path, and therefore the makespan, while the other rules are used to maximise the assignment effectiveness.

**High SP:** In this case, the network structures are more serial. Therefore, the number of critical activities increases and the usage percentage of the HAD rule increases too.

No substantial differences were found for the selection of the resource priority rules over different numbers of available resources $|R|$.

### 6.4. Analysis of the resource parameters

In this section, the ideal workforce characteristics are analysed. This is done by comparing 25 project-specific workforces for each project available in the dataset. These workforces were created using the $SS_j$ and *SA* parameters, both going from 0 to 1 in steps of 0.25, resulting in 25 different workforces. Various graphs comparing the results for the different parameters are displayed in Fig. 7. The graphs show the impact of the *AvgSS* ($\sum_j \frac{SS_j}{|S|}$) and the *SA* on the makespan of the project. Furthermore, a comparison is made based on the SP. Highly parallel ($SP = 0.1$) and highly serial ($SP = 0.9$) network structures are compared to instances with an SP equal to 0.5.

Fig. 7 shows that the makespan is better when the multi-skilledness in the workforce is lower (high *SA*). For projects with a high *SP* the impact of the *SA* is very limited. A high amount of available skills (high *AvgSS*) also has a positive effect on the makespan. However, the lowest makespan can in most cases be found with a lower *AvgSS*. This is due to the fact that if the *AvgSS*

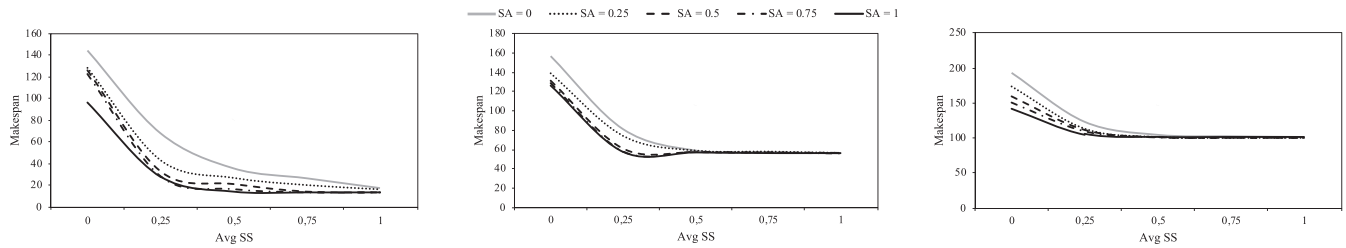


**Fig. 7.** Impact of parameters on objective values (Left to right: $SP = 0.1$, $SP = 0.5$, $SP = 0.9$).

lies close to 1, some of the available skills might be redundant. Only very low *AvgSS*-values have an impact on the makespan for projects with a high *SP*, which implies that the number of redundant skills increases with the *SP*.

## 7. Conclusion

In this paper, the MSRCPSP with breadth and depth is discussed. This scheduling problem is an extension to the well-known RCPSP, and states that resources can now master more than one skill. Hence, resources can be assigned in a more flexible way to the activities. In this work, the objective is to investigate the properties and characteristics of an efficient workforce under a makespan objective.

To solve this problem we developed a genetic algorithm. The representation and crossovers used in this GA are adapted to the specific characteristics of this MSRCPSP. Furthermore, three local searches for the scheduling and assignment of projects are presented. We show that our representation outperforms a previously published representation and that an aggregate procedure generates even better solutions. Furthermore, we explain that the depth of the skills only has a substantial impact on the schedule and assignment for predominantly parallel projects. The workforce size and the breadth of the resources is dependent on the objective (makespan) and on the seriality of the project. We demonstrated that parallel projects benefit more from flexible resources with a higher breadth, as they can be assigned more efficiently. For serial projects, more flexibility has no substantial impact, so the focus should be on maximising depth of the assigned resources in order to minimise the duration of the individual activities.

Future research in this area could focus on the integration of learning in the MSRCPSP. Since efficiencies of workers tend to be a very dynamic concept, it can be a promising addition to the MSRCPSP to incorporate an adjustable depth that is affected by learning.

## Acknowledgments

We acknowledge the support provided by the Nationale Bank van Belgi (NBB) and the Bijzonder Onderzoeksfonds (BOF) for the project, under contract number BOF12GOA021.

The computational resources (Stevin Supercomputer Infrastructure) and services used in this work were provided by the VSC (Flemish Supercomputer Center), funded by Ghent University, FWO and the Flemish Government - department EWI.

## Appendix A. Technical explanation for the formula for the adjusted processing time (appendix)

*Introduction*

In Section 3 of the paper, the adjusted activity duration is based on the depth of the skills of the resources assigned to this activity. This calculation can be done under various assumptions, taking into account that an activity might require different skills. In order to show why we have implemented Eq. (2) in this paper, we present in this appendix four different versions to calculate the activity duration, we compare them and show their advantages and disadvantages. Even though our procedure can cope with all four versions to measure the impact of the depth of resources on the activity duration, we show that the second version, as implemented in our final procedure, might be the most realistic one for use in this and future research studies.

*Adjusted processing time: 4 versions*

*Version 1 (average depth with reciprocal relation):* In this version, a reciprocal relation between the average depth of the assigned resources $\overline{d_i}$ and the adjusted processing time $p_i^a$ of activity $i$:

$$p_i^a = p_i \cdot \frac{1}{\overline{d_i}} \tag{A.1}$$

$$\overline{d_i} = \frac{\sum_{j \in S} \sum_{k \in R} d_{jk} \cdot x_{ijk}}{\sum_{j \in S} r_{ij}} \tag{A.2}$$

The depth $d_{jk}$ of all resources $k \in R$ assigned to activity $i$ is averaged. Hence, all resources impact each other, even if they perform different skills. A $\overline{d_i}$ equal to 0.5 will result in an adjusted processing time that is twice as long as the standard processing time.

*Version 2 (average depth per skill with reciprocal relation):* The relation between the depth $d_{jk}$ and the adjusted processing time $p_i^a$ is slightly different in the papers of Heimerl and Kolisch (2010), Kolisch and Heimerl (2012) and Walter and Zimmermann (2016). In these papers the skill requirements $r_{ij}^{unit}$ are represented in workload units and a resource $k$ performing a skill $j$ only has an impact on the required workload of skill $j$. If $w_{ijk}$ represents the number of time units that resource $k$ works on activity $i$ with skill $j$, we can change equation (6) on page 10 of this paper to the following formula:

$$\sum_{k \in R} d_{jk} \cdot w_{ijk} = r_{ij}^{unit} \tag{A.3}$$

In this case, resource $k$ only affects the required workload for skill $j$ and the duration depends on the skill $j$ which takes the longest to execute. In order to create an equivalent effect that can deal with the unit skill requirements $r_{ij}$ of our new dataset, we can split up Eq. (A.3) into the original equation (6) on page 10 of the paper and the new Eq. (A.4) that specifies the new adjusted processing time $p_i^a$. Therefore. the average depth of the assigned resources $\overline{d_{ij}}$ is now calculated per skill $j$ for each activity $i$. To determine the adjusted processing time $p_i^a$, we take the reciprocal of the minimum average depth of the assigned resources $\min_j(\overline{d_{ij}})$ and multiply this by the standard processing time $p_i$:

$$p_i^a = p_i \cdot \frac{1}{\min_j(\overline{d_{ij}})} \tag{A.4}$$

$$\overline{d_{ij}} = \frac{\sum_{k \in R} d_{jk} \cdot x_{ijk}}{r_{ij}} \tag{A.5}$$

**Table A.10**
Mathematical model details (single-skilled).

| | $d_{11}$ | $d_{12}$ | $\overline{d_i}$ | $\overline{d_{i1}}$ | $p^a_{i,1}$ | $p^a_{i,2}$ | $p^a_{i,3}$ | $p^a_{i,4}$ |
|---|---|---|---|---|---|---|---|---|
| Combination 1 | 1 | 1 | 1 | 1 | 5 | 5 | 5 | 5 |
| Combination 2 | 1.5 | 0.5 | 1 | 1 | 5 | 5 | 3.33 | 10 |
| Combination 3 | 0.5 | 1 | 0.75 | 0.75 | 6.66 | 6.66 | 5 | 10 |

**Table A.11**
Mathematical model details (multi-skilled).

| | $d_{11}$ | $d_{12}$ | $d_{23}$ | $\overline{d_i}$ | $\overline{d_{i1}}$ | $\overline{d_{i2}}$ | $p^a_{i,1}$ | $p^a_{i,2}$ | $p^a_{i,3}$ | $p^a_{i,4}$ |
|---|---|---|---|---|---|---|---|---|---|---|
| Combination 1 | 1 | 1 | 1 | 1 | 1 | 1 | 5 | 5 | 5 | 5 |
| Combination 2 | 1.5 | 0.5 | 0.5 | 0.833 | 1 | 0.5 | 6 | 10 | 3.33 | 10 |
| Combination 3 | 0.5 | 1 | 1.5 | 1 | 0.75 | 1.5 | 5 | 6.66 | 3.33 | 10 |

Similarly to the previously mentioned papers, the adjusted processing time $p^a_i$ is only affected by the skill that is performed the slowest, i.e. the skill with the lowest assigned average depth $\min_j(\overline{d_{ij}})$. If this value is equal to 0.5 the adjusted processing time will take twice the standard processing time.

*Version 3 (best resource with reciprocal relation):* The best case version only takes the depth $d_{jk}$ of the most efficient resource $k$ into account. This version is the lower bound on the relation between the depth $d_{jk}$ and the adjusted processing time $p^a_i$.

$$p^a_i = p_i \cdot \frac{1}{\max_j(\max_k(d_{jk} \cdot x_{ijk}))} \tag{A.6}$$

*Version 4 (worst resource with reciprocal relation):* The worst case version only takes the depth $d_{jk}$ of the least efficient resource $k$ into account. This version is the upper bound on the relation between the depth $d_{jk}$ and the adjusted processing time $p^a_i$. Furthermore, $u_{ij}$ is equal to 1 if $r_{ij} > 0$ and 0 otherwise.

$$p^a_i = p_i \cdot \frac{1}{\min_j(\max_k(d_{jk} \cdot x_{ijk} + (1 - u_{ij}) \cdot d^{max})} \tag{A.7}$$

*Comparison/Example*

Assume we have an activity $i$ that requires 2 resources of skill 1, i.e. $r_{i1} = 2$, and has a standard duration of 5, then following combinations of resources and adjusted durations are possible.

From Table A.10 it is clear that version 1 and 2 give the same results for the single-skilled case, because $\overline{d_i} = \min_j(\overline{d_{ij}})$. Furthermore, the best (worst) case always results in a lower (higher) or equal adjusted processing time $p^a_i$ in comparison to version 1 and 2.

For a second example, we add a second skill requirement to this example, and make it multi-skilled. Assume that we now require an extra resource for skill 2, i.e. $r_{i2} = 1$, then the following combinations can occur.

From Table A.11, we can see that version 1 and 2 can now result in different solutions. Moreover, $p^a_{i,2}$ will always be higher than or equal to $p^a_{i,1}$, leading to following inequalities:

$$p^a_{i,3} \leq p^a_{i,1} \leq p^a_{i,2} \leq p^a_{i,4} \tag{A.8}$$

*Conclusion*

The biggest difference between these versions is the rate at which the depth of the resources can impact each other. In version 1, collaboration crosses the boundaries of the skills and in version 2 collaboration stays within the performed skill. In the best case version, we assume that all resources benefit from working with the most efficient resource and in the worst case version there is no collaboration at all and the processing time depends on the least efficient resource.

For each version multiple combinations of depth can result in the same adjusted processing time. Since the adjusted processing time is only affected by the depth of one resource, it is clear that the most equivalent combinations exist for version 3 and 4. The number of equivalent combinations is lower for version 2 and the lowest for version 1.

We conclude that the most realistic case occurs if resources can only be influenced by other resources that perform the same skill, as was intended in Heimerl and Kolisch (2010), Kolisch and Heimerl (2012) and Walter and Zimmermann (2016). Therefore, we have chosen to incorporate version 2 in our paper.

## References


Alba, E., & Chicano, J. F. (2007). Software project management with GAs. *Information Sciences, 177*(11), 2380–2401.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2016). Priority-based heuristics for the multi-skill resource constrained project scheduling problem. *Expert Systems with Applications, 57*, 91–103.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2018a). A biased random-key genetic algorithm for the project scheduling problem with flexible resources. *TOP*, 1–26.

Almeida, B. F., Correia, I., & Saldanha-da Gama, F. (2018b). Modeling frameworks for the multiskill resource-constrained project scheduling problem: A theoretical and empirical comparison. *International Transactions in Operational Research*.

Bedworth, D. D., & Bailey, J. E. (1982). *Integrated production control systems: Management, analysis, design*. John Wiley & Sons, Inc..

Bellenguez, O., & Néron, E. (2004). Lower bounds for the multi-skill project scheduling problem with hierarchical levels of skills. In *Proceedings of the international conference on the practice and theory of automated timetabling* (pp. 229–243). Springer.

Bellenguez-Morineau, O., & Néron, E. (2007). A branch-and-bound method for solving multi-skill project scheduling problem. *RAIRO-Operations Research, 41*(2), 155–170.

Blazewicz, J., Lenstra, J. K., & Kan, A. R. (1983). Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics, 5*(1), 11–24.

Brusco, M. J., & Johns, T. R. (1998). Staffing a multiskilled workforce with varying levels of productivity: An analysis of cross-training policies. *Decision Sciences, 29*(2), 499–515.

Cai, X., & Li, K. N. (2000). A genetic algorithm for scheduling staff of mixed skills under multi-criteria. *European Journal of Operational Research, 125*(2), 359–369.

Correia, I., & Saldanha-da Gama, F. (2014). The impact of fixed and variable costs in a multi-skill project scheduling problem: An empirical study. *Computers & Industrial Engineering, 72*, 230–238.

Correia, I., Lampreia-Lourenço, L., & Saldanha-da Gama, F. (2012). Project scheduling with flexible resources: Formulation and inequalities. *OR Spectrum, 34*(3), 635–663.

Datta, D. K., Guthrie, J. P., Basuil, D., & Pandey, A. (2010). Causes and effects of employee downsizing: A review and synthesis. *Journal of Management, 36*(1), 281–348.

Gomar, J. E., Haas, C. T., & Morton, D. P. (2002). Assignment and allocation optimization of partially multiskilled workforce. *Journal of Construction Engineering and Management, 128*(2), 103–109.

Gutjahr, W. J., Katzensteiner, S., Reiter, P., Stummer, C., & Denk, M. (2010). Multi-objective decision analysis for competence-oriented project portfolio selection. *European Journal of Operational Research, 205*(3), 670–679.

Haas, C. T., Rodriguez, A. M., Glover, R., & Goodrum, P. M. (2001). Implementing a multiskilled workforce. *Construction Management & Economics, 19*(6), 633–641.

Hartmann, S. (1998). A competitive genetic algorithm for resource-constrained project scheduling. *Naval Research Logistics (NRL), 45*(7), 733–750.

Hartmann, S., & Briskorn, D. (2010). A survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research, 207*(1), 1–14.
Hegazy, T., Shabeeb, A. K., Elbeltagi, E., & Cheema, T. (2000). Algorithm for scheduling with multiskilled constrained resources. *Journal of Construction Engineering and Management, 126*(6), 414–421.
Heimerl, C., & Kolisch, R. (2010). Scheduling and staffing multiple projects with a multi-skilled workforce. *OR Spectrum, 32*(2), 343–368.
Hopp, W. J., Tekin, E., & Van Oyen, M. P. (2004). Benefits of skill chaining in serial production lines with cross-trained workers. *Management Science, 50*(1), 83–98.
Inman, R. R., Jordan, W. C., & Blumenfeld, D. E. (2004). Chained cross-training of assembly line workers. *International Journal of Production Research, 42*(10), 1899–1910.
Kolisch, R., & Heimerl, C. (2012). An efficient metaheuristic for integrated scheduling and staffing it projects based on a generalized minimum cost flow network. *Naval Research Logistics, 59*(2), 111–127.
Kolisch, R., Sprecher, A., & Drexl, A. (1995). Characterization and generation of a general class of resource-constrained project scheduling problems. *Management science, 41*(10), 1693–1703.
Labro, E., & Vanhoucke, M. (2008). Diversity in resource consumption patterns and robustness of costing systems to errors. *Management Science, 54*(10), 1715–1730.
Li, H., & Womer, K. (2009). Scheduling projects with multi-skilled personnel by a hybrid MILP/CP benders decomposition algorithm. *Journal of Scheduling, 12*(3), 281.
Li, K. Y., & Willis, R. J. (1992). An iterative scheduling technique for resource–constrained project scheduling. *European Journal of Operational Research, 56*(3), 370–379.
Liu, S.-S., & Wang, C. J. (2012). Optimizing linear project scheduling with multi--skilled crews. *Automation in Construction, 24*, 16–23.
Mansfield, R. S. (1996). Building competency models: Approaches for HR professionals. *Human Resource Management, 35*(1), 7–18.
Martignago, M., Battaïa, O., & Battini, D. (2017). Workforce management in manual assembly lines of large products: A case study. *IFAC-PapersOnLine, 50*(1), 6906–6911.
McCune, J. C. (1994). On the train gang. *Management Review, 83*(10), 57.
Montoya, C., Bellenguez-Morineau, O., Pinson, E., & Rivreau, D. (2014). Branch-and-price approach for the multi-skill project scheduling problem. *Optimization Letters, 8*(5), 1721–1734.
Nembhard, D. A., & Uzumeri, M. V. (2000). Experiential learning and forgetting for manual and cognitive tasks. *International Journal of Industrial Ergonomics, 25*(4), 315–326.
Néron, E. (2002). Lower bounds for the multi-skill project scheduling problem. *Proceeding of the eighth international workshop on project management and scheduling* (pp. 274–277).
Pinker, E. J., & Shumsky, R. A. (2000). The efficiency-quality trade-off of cross–trained workers. *Manufacturing & Service Operations Management, 2*(1), 32–48.
Schultz, K. L., McClain, J. O., & Thomas, L. J. (2003). Overcoming the dark side of worker flexibility. *Journal of Operations Management, 21*(1), 81–92.
Valls, V., Pérez, A., & Quintanilla, S. (2009). Skilled workforce scheduling in service centres. *European Journal of Operational Research, 193*(3), 791–804.
Van Peteghem, V., & Vanhoucke, M. (2010). A genetic algorithm for the preemptive and non-preemptive multi-mode resource-constrained project scheduling problem. *European Journal of Operational Research, 201*(2), 409–418.
Vanhoucke, M., Coelho, J., Debels, D., Maenhout, B., & Tavares, L. V. (2008). An evaluation of the adequacy of project network generators with systematically sampled networks. *European Journal of Operational Research, 187*(2), 511–524.
Walter, M., & Zimmermann, J. (2016). Minimizing average project team size given multi-skilled workers with heterogeneous skill levels. *Computers & Operations Research, 70*, 163–179.
Yannibelli, V., & Amandi, A. (2011). A knowledge-based evolutionary assistant to software development project scheduling. *Expert Systems with Applications, 38*(7), 8403–8413.